



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Taller 2: Análisis sintáctico, semántico y generación de código en YACC

Álvaro Camilo Torres Rodriguez - alvtorresro@unal.edu.co
Juan David Cárdenas Galvis - jcardenasgal@unal.edu.co
Laura Vanesa Alvarez Lafont - lalvarezla@unal.edu.co
Nicolás Andrés Bolaños Fernandez - nbolanosf@unal.edu.co
Santiago Andrés Fetecua Pulgarín - sfetecua@unal.edu.co
Sergio Nicolás Correa Escobar - secorreae@unal.edu.co

Docente

Jorge Eduardo Ortiz Triviño

Universidad Nacional de Colombia
Departamento de Ingeniería de Sistemas e Industrial
Lenguajes de programación
Bogotá D.C., Colombia
2026

Contenidos

1. Marco Teórico.....	3
2. Descripción del Problema.....	8
3. Justificación del Problema.....	9
4. Diseño de Solución.....	11
5. Código Fuente.....	25
6. Manual de Usuario.....	26
a. Requisitos.....	26
b. Ejecución e Interfaz Principal.....	26
c. Gestión y Edición de Memoria RAM.....	27
d. Procesamiento de Lenguaje.....	28
e. Consideraciones.....	39
7. Manual Técnico.....	40
a. Introducción y Visión General del Sistema.....	40
b. Preprocesador.....	40
c. Ensamblador.....	42
d. Enlazador-Cargador.....	44
e. Módulo de análisis léxico del compilador.....	45
f. Análisis Sintáctico.....	47
g. Análisis Semántico.....	50
h. Generador de Código.....	55
i. Conclusiones.....	56
8. Experimentación y Análisis de Resultados.....	57
9. Bibliografía.....	69
10. Apéndice.....	70

1. Marco Teórico

Un Sistema de Procesamiento de Lenguaje (SPL) es un conjunto de herramientas que permiten transformar un programa escrito en un lenguaje fuente en una representación ejecutable por una máquina. Este proceso involucra múltiples etapas, cada una con responsabilidades específicas dentro de la cadena de traducción.

Un SPL clásico se compone de:

- **Preprocesador**
- **Compilador**
- **Ensamblador**
- **Enlazador-Cargador**

Cada uno de estos módulos contribuye a la transformación progresiva del código, pasando de una representación de alto nivel a código ejecutable.

Preprocesador: El preprocesador es una etapa previa a la compilación encargada de realizar transformaciones textuales sobre el código fuente. No analiza la semántica del programa, sino que aplica reglas de sustitución y expansión.

Entre sus funciones principales se encuentran:

- Inclusión de archivos (`#include`)
- Definición de macros (`#define`)
- Sustitución de constantes simbólicas

El uso de herramientas como PLY (python lex and yacc) permite definir reglas basadas en expresiones regulares para reconocer y transformar estas directivas.

Compilador: El compilador es el componente central del SPL. Su función es traducir un programa en lenguaje de alto nivel a un lenguaje de nivel inferior (generalmente código intermedio o ensamblador).

Se estructura típicamente en varias fases:

Análisis Léxico: El análisis léxico es la primera fase del compilador. Se encarga de transformar una secuencia de caracteres en una secuencia de tokens.

Un token representa una unidad significativa del lenguaje, como:

- Identificadores
- Palabras reservadas
- Operadores
- Literales

Este proceso se fundamenta en el uso de expresiones regulares y autómatas finitos, que permiten reconocer patrones dentro del texto. Las categorías léxicas definen los tipos de tokens que el compilador reconoce. Algunas categorías típicas incluyen:

- Identificadores: nombres de variables, funciones
- Palabras reservadas: if, while, return
- Constantes: números enteros, reales, cadenas
- Operadores: +, -, *, /
- Delimitadores: ;, ,, {, }

Cada categoría se define mediante patrones formales, usualmente expresiones regulares. Las expresiones regulares permiten describir formalmente los patrones que conforman los tokens. Estas definiciones son la base para la construcción de analizadores léxicos automatizados.

Análisis Sintáctico

El análisis sintáctico, también conocido como parsing, es la fase del compilador encargada de verificar que la secuencia de tokens generada por el análisis léxico cumple con las reglas gramaticales del lenguaje.

Esta etapa se basa en una gramática formal, generalmente una gramática libre de contexto (GLC), que define la estructura válida de los programas. El analizador sintáctico construye una representación estructurada del programa, como:

- Árbol de derivación
- Árbol sintáctico abstracto (AST)

El análisis sintáctico permite detectar errores como: el uso incorrecto de delimitadores (paréntesis, llaves), estructuras mal formadas, o el orden inválido de los elementos del lenguaje

Esta fase es fundamental, ya que establece la estructura jerárquica del programa y sirve como base para el análisis semántico posterior.

Análisis Semántico

El análisis semántico es la fase del compilador encargada de verificar que el programa, además de ser sintácticamente correcto, tenga un significado válido dentro del contexto del lenguaje.

En esta etapa se trabaja sobre el árbol sintáctico generado previamente y se utilizan estructuras como la tabla de símbolos para validar información contextual.

Entre las principales tareas del análisis semántico se encuentran:

- Verificación de tipos (type checking)
- Validación de declaraciones de variables
- Control de alcance (scope)
- Compatibilidad entre operaciones y operandos
- Detección de usos indebidos de identificadores

Por ejemplo, el análisis semántico permite detectar errores como el uso de variables no declaradas, las asignaciones entre tipos incompatibles, y las llamadas a funciones con número incorrecto de argumentos

A diferencia del análisis léxico y sintáctico, que se basan en estructuras formales, el análisis semántico requiere conocimiento del contexto y las reglas del lenguaje, lo que lo convierte en una etapa más compleja.

Esta fase garantiza la coherencia lógica del programa antes de proceder a etapas posteriores como la generación de código.

Ensamblador: El ensamblador traduce código en lenguaje ensamblador a código máquina. Este proceso implica desde la traducción de instrucciones simbólicas a códigos binarios, pasando por la resolución de direcciones hasta el manejo de etiquetas

En el contexto del proyecto, el ensamblador se implementa también mediante PLY tanto para su lexer como para su parser, enfocándose en el reconocimiento de instrucciones y operandos.

Enlazador-Cargador: El enlazador-cargador constituye una etapa fundamental dentro del Sistema de Procesamiento de Lenguaje, encargada de integrar y preparar el programa

final para su ejecución en memoria. En primer lugar, el enlazador (linker) se ocupa de combinar múltiples módulos objeto generados de manera independiente durante las fases previas de compilación o ensamblado. Este proceso implica la resolución de referencias externas, es decir, la asociación correcta entre identificadores definidos en distintos módulos, garantizando así la coherencia global del programa. Asimismo, el enlazador construye una única unidad ejecutable, organizando las distintas secciones del programa (código, datos, etc.) y estableciendo las direcciones relativas necesarias para su correcto funcionamiento.

Posteriormente, el cargador (loader) asume la responsabilidad de ubicar el programa enlazado en la memoria principal del sistema. Para ello, realiza procesos de relocalización, ajustando las direcciones de memoria según la posición real en la que el programa será cargado, y prepara el entorno necesario para su ejecución, incluyendo la inicialización de registros y estructuras auxiliares. Esta fase resulta esencial para garantizar que el programa pueda ejecutarse correctamente en el entorno destino, independientemente de las condiciones en las que fue compilado originalmente. En conjunto, el enlazador-cargador permite la modularidad, reutilización de código y correcta transición entre la representación intermedia y la ejecución efectiva del programa.

Paradigma Imperativo y Tipos de Datos Abstractos (TDA): El desarrollo del compilador propuesto en este proyecto se fundamenta en el paradigma imperativo, el cual se caracteriza por la descripción explícita de una secuencia de instrucciones que modifican el estado del sistema a lo largo del tiempo. En este enfoque, el programa se concibe como una serie de operaciones que actúan sobre variables almacenadas en memoria, utilizando estructuras de control que determinan el flujo de ejecución, tales como la secuencia, la selección (mediante construcciones condicionales como if-else) y la iteración (a través de estructuras como while o for). Este paradigma resulta particularmente adecuado para la construcción de compiladores, dado que permite un control preciso sobre el manejo de datos, la memoria y las estructuras internas necesarias para el procesamiento del lenguaje.

Adicionalmente, el diseño del sistema incorpora el uso de Tipos de Datos Abstractos (TDA), los cuales permiten modelar de manera estructurada y modular las entidades necesarias para el funcionamiento del compilador. Los TDA facilitan la encapsulación de datos junto con las operaciones que se pueden realizar sobre ellos, promoviendo la claridad, reutilización y mantenibilidad del código. En el contexto del compilador, estos pueden incluir estructuras como listas o secuencias para el manejo de tokens, pilas para la implementación de algoritmos de análisis sintáctico, y tablas de símbolos para el almacenamiento y consulta de información relevante sobre identificadores. Asimismo, el lenguaje objetivo contempla el manejo de variables, arreglos y matrices, lo que implica la

necesidad de definir representaciones internas adecuadas y mecanismos para su manipulación. En conjunto, la combinación del paradigma imperativo con el uso de TDA proporciona una base sólida y organizada para la implementación eficiente y escalable del sistema de procesamiento de lenguaje.

2. Descripción del Problema

En el desarrollo de compiladores modernos, uno de los retos principales consiste en integrar correctamente las distintas fases de procesamiento de un lenguaje dentro de una arquitectura coherente y extensible. Aunque cada etapa, como el análisis léxico, sintáctico, semántico o la generación de código, puede estudiarse de manera aislada, el verdadero desafío radica en lograr que todas trabajen de forma coordinada, compartiendo estructuras internas y manteniendo consistencia durante todo el flujo de traducción.

El presente taller aborda el diseño e implementación de un Sistema de Procesamiento de Lenguaje (SPL) para la arquitectura educativa CacaoCore-64, incorporando no solamente las etapas iniciales del procesamiento, sino también mecanismos de validación sintáctica, verificación semántica y generación de código intermedio y ensamblador.

El sistema implementado permite transformar programas escritos en el lenguaje CHOCO en representaciones progresivamente más cercanas a la máquina, pasando por procesos de:

- Preprocesamiento.
- Análisis léxico.
- Análisis sintáctico.
- Análisis semántico.
- Generación de ensamblador.
- Ensamblado.
- Enlazado y carga.

Desde una perspectiva de ingeniería de software, el problema implica definir formalmente el lenguaje fuente, construir una gramática consistente, validar reglas contextuales mediante tablas de símbolos y producir código compatible con la arquitectura CacaoCore-64.

Adicionalmente, el proyecto requiere mantener una arquitectura modular y extensible, donde cada componente opere de manera desacoplada pero interoperable. Esto permite reutilizar estructuras internas como tablas de símbolos, árboles sintácticos abstractos y representaciones intermedias a lo largo de todo el flujo de compilación.

3. Justificación del Problema

La construcción de un Sistema de Procesamiento de Lenguaje en sus fases iniciales responde a la necesidad de consolidar una comprensión integral del proceso de traducción de programas, más allá del estudio aislado de sus componentes. En particular, la implementación de módulos como el preprocesador, el ensamblador, el enlazador-cargador y el analizador léxico permite entender cómo un programa fuente es transformado progresivamente mediante una cadena de procesamiento estructurada, donde cada etapa cumple un rol específico dentro de un flujo bien definido.

Desde una perspectiva técnica, este enfoque resulta fundamental, ya que obliga a modelar cada componente como un sistema de transformación formal, con especificaciones claras de entrada, salida y reglas de operación. La utilización de herramientas basadas en expresiones regulares y generadores de analizadores léxicos introduce al estudiante en prácticas propias del diseño de compiladores reales, donde la precisión en la definición del vocabulario, la correcta clasificación de tokens y la consistencia en el reconocimiento de patrones son aspectos críticos para garantizar el correcto funcionamiento del sistema.

Adicionalmente, la implementación de estos módulos permite enfrentar problemáticas propias del desarrollo de herramientas de traducción, tales como la expansión de macros, la resolución de referencias, la estructuración de información simbólica y la correcta segmentación del código fuente. Estos desafíos no solo refuerzan conceptos teóricos de lenguajes formales, sino que también desarrollan habilidades de diseño modular, abstracción y manejo de complejidad, al requerir que cada componente sea coherente tanto internamente como en su interacción con los demás.

Por otro lado, el desarrollo progresivo del SPL en fases permite construir una arquitectura escalable, en la cual las etapas implementadas en este taller constituyen la base para fases posteriores del compilador, como el análisis sintáctico, semántico y la generación de código. Este enfoque incremental facilita la validación temprana de cada componente y promueve una comprensión más profunda del flujo completo de compilación, desde el tratamiento inicial del código hasta su preparación para ejecución.

En conjunto, este ejercicio no solo cumple un propósito académico inmediato, sino que aproxima al estudiante a la ingeniería de compiladores desde una perspectiva práctica, fortaleciendo competencias clave en el diseño de sistemas, el uso de herramientas formales y la construcción de soluciones robustas dentro del ámbito del procesamiento de lenguajes.

La construcción de un Sistema de Procesamiento de Lenguaje completo permite consolidar de manera práctica los fundamentos teóricos asociados al diseño de compiladores y arquitecturas de traducción de lenguajes. Más allá de implementar

herramientas independientes, el desarrollo de un SPL integrado permite comprender cómo interactúan las distintas fases de compilación para transformar un programa de alto nivel en código ejecutable.

La inclusión de análisis sintáctico y semántico dentro del flujo del compilador fortalece significativamente la robustez del sistema, ya que no solo se verifica la estructura formal del lenguaje, sino también la coherencia lógica y contextual de los programas escritos por el usuario.

En particular, el sistema implementado permite:

- Validar declaraciones y usos de identificadores.
- Gestionar ámbitos y tablas de símbolos.
- Verificar compatibilidad de tipos.
- Detectar redeclaraciones.
- Validar llamadas a funciones y parámetros.
- Verificar estructuras y acceso a miembros.
- Generar representaciones intermedias consistentes.
- Traducir instrucciones a ensamblador compatible con CacaoCore-64.

Desde el punto de vista académico, el proyecto aproxima al estudiante a técnicas reales utilizadas en compiladores modernos, incluyendo el uso de herramientas basadas en LEX/YACC mediante PLY, construcción de ASTs, validación semántica contextual y traducción dirigida por sintaxis.

Asimismo, el enfoque modular adoptado permite que el sistema sea escalable y mantenible, facilitando futuras extensiones del lenguaje y nuevas optimizaciones sobre la generación de código.

4. Diseño de Solución

La construcción de nuestra solución se realizó de forma modular, garantizando que cada sección funcionara de forma independiente para luego integrarlas y montar su funcionamiento en el CacaoCore-64, una máquina emulada de 64 bits diseñada previamente por el mismo equipo.

Preprocesador

Se diseñó un módulo independiente basado en una arquitectura de procesamiento lineal en dos fases. En la primera fase se utiliza el lexer PLY para tokenizar el texto fuente y clasificar cada fragmento antes de interpretarlo, evitando ambigüedades entre directivas, comentarios y código arbitrario. El preprocesador soporta directivas `#include`, `#define` (incluyendo macros con parámetros), condicionales de compilación (`#if`, `#elif`, `#else`, `#endif`), y un sistema integral de librerías externas con soporte para funciones externas (externs).

Para el manejo de macros se diseñó una clase dedicada (Macro) que distingue entre macros simples y macros con parámetros, aplicando la expansión mediante sustitución textual en múltiples pasadas iterativas con un límite configurable para prevenir expansión infinita. Las inclusiones de archivos se resuelven de forma recursiva manteniendo una pila de archivos activos para detectar ciclos y un conjunto de archivos ya procesados para evitar inclusiones duplicadas, con validación de rutas para prevenir accesos fuera del directorio de bibliotecas. El resultado final es una estructura `PreprocessResult` que separa el texto procesado del mapa de trazabilidad de líneas, desacoplando así el preprocesador del resto del compilador y permitiendo que cada componente posterior opere de forma independiente.

Macros con Parámetros: El preprocesador soporta macros que aceptan parámetros formales, permitiendo una mayor flexibilidad en la reutilización de código.

La expansión se realiza mediante sustitución textual en múltiples pasadas iterativas, con un límite configurable (por defecto 50) para prevenir expansiones infinitas.

Condicionales de Compilación: Se implementan directivas condicionales que permiten compilar código condicionalmente según se definan símbolos específicos. Esto facilita la compilación selectiva para diferentes configuraciones o plataformas.

Sistema de Librerías: Las inclusiones de archivos se resuelven recursivamente, manteniendo una pila de archivos activos para detectar ciclos y un conjunto de archivos ya procesados para evitar inclusiones duplicadas. Se mantiene una jerarquía de directorios de bibliotecas para validar rutas y prevenir accesos

fuera del directorio permitido.

Compilador (Analizador léxico)

Para este módulo se empleó un paradigma orientado a objetos. La idea fue construir una clase Compiler capaz de realizar (proyectando a futuro) todas las etapas de compilación, es decir, los análisis léxico, sintáctico y semántico. Una instancia de esta clase se comunica con la interfaz encargada del SPL.

El analizador léxico, o lexer, contiene las siguientes declaraciones explícitas para el lenguaje CHOCO

Vocabulario: {let, set, func, if, otherwise, asLongAs, for, in, deliver, show, oops, int, float, text, bool, indeed, nope, nothing, mold, ohmy, summon} U Enteros U Flotantes U Identificadores U Operadores U Delimitadores U Otros

donde los patrones para los subconjuntos enunciados son:

- Enteros: $[0-9]^+$
- Flotantes: $[0-9]^+.[0-9]^+$
- Identificadores: $[a-zA-Z_][a-zA-Z_0-9]^*$
- Operadores: $\{+, -, *, /, \%, ==, !=, >=, <=, <, >, =\}$
- Delimitadores: $\{ (,), \{, \}, [,], :, ,, . \}$
- Otros: $\{ \text{whitespace}, \backslash t, \backslash n, // \}$

Categorías gramaticales: Las categorías gramaticales corresponden a la clasificación de los tokens según su función dentro de la estructura del lenguaje.

- **Palabras reservadas:** [IF, OTHERWISE, ASLONGAS, FOR, IN, LET, SET, FUNC, DELIVER, SHOW, OOPS, MOLD, OHMY, SUMMON]
- **Tipos nativos:** [INT_TYPE, FLOAT_TYPE, TEXT_TYPE, BOOL_TYPE]
- **Literales:** [NUM, STRING, INDEED, NOPE, NOTHING]
- **Identificadores:** [ID]
- **Operadores:** [PLUS, MINUS, TIMES, DIVIDE, MOD, EQ, NEG, LT, GT, LEQ, GEQ, ASSIGN, OR, AND, NOT, XOR]
- **Delimitadores:** [LPAREN, RPAREN, LBRACE, RBRACE, LBRACKET, RBRACKET, COLON, COMMA, DOT]
- **No significativos:** No tienen un token, pero son los elementos ignorados como los whitespaces, saltos de línea, y comentarios

Definiciones auxiliares: Para las posteriores definiciones de patrones

LETRA = $[a-zA-Z]$

DIGITO = [0-9]
GUION_BAJO = [_]

Patrones de expresiones regulares:

IF = "if"
OTHERWISE = "otherwise"
ASLONGAS = "asLongAs"
FOR = "for"
IN = "in"
LET = "let"
SET = "set"
FUNC = "func"
DELIVER = "deliver"
SHOW = "show"
OOPS = "oops"
MOLD = "mold"
OHMY = "ohmy"
SUMMON = "summon"
INT_TYPE = "int"
FLOAT_TYPE = "float"
TEXT_TYPE = "text"
BOOL_TYPE = "bool"
INDEED = "indeed"
NOPE = "nope"
NOTHING = "nothing"

NUM = DIGITO+ ("." DIGITO+)?
STRING = " \" [^\n]* \" "
ID = (LETRA | GUION_BAJO) (LETRA | DIGITO | GUION_BAJO)*

PLUS = "+"
MINUS = "-"
TIMES = "*"
DIVIDE = "/"
MOD = "%"
EQ = "=="
NEG = "!="
LT = "<"
GT = ">"
LEQ = "<="

```
GEQ = ">="
ASSIGN = "="
LPAREN = "("
RPAREN = ")"
LBRACE = "{"
RBRACE = "}"
LBRACKET = "["
RBRACKET = "]"
COLON = ":"
COMMA = ","
DOT = "."

WHITESPACE = [ \t ]+
NEWLINE = \n+
COMMENT = '/*[^n]*'
```

Análisis Sintáctico

El análisis sintáctico fue implementado mediante reglas yacc utilizando gramáticas libres de contexto.

Esta etapa verifica que la secuencia de tokens cumpla con las estructuras válidas del lenguaje CHOCO y construye simultáneamente el Árbol Sintáctico Abstracto (AST).

La gramática soporta construcciones como:

Declaraciones de variables.

Asignaciones.

Funciones.

Parámetros.

Llamadas.

Expresiones aritméticas.

Expresiones booleanas.

Condicionales.

Ciclos.

Retornos.

Acceso a estructuras.

Definición de moldes.

La precedencia de operadores fue definida explícitamente para evitar ambigüedades gramaticales, permitiendo manejar correctamente operaciones aritméticas, lógicas y relacionales.

El parser implementa recuperación básica ante errores sintácticos, identificando:

- Delimitadores faltantes.
- Expresiones incompletas.
- Uso inválido de operadores.
- Estructuras mal formadas.
- Tokens inesperados.
- Árbol Sintáctico Abstracto (AST)

El compilador construye una representación jerárquica del programa mediante un AST compuesto por nodos especializados.

Cada nodo representa una construcción del lenguaje y encapsula:

- Tipo de instrucción.
- Información contextual.
- Tipos inferidos.
- Hijos del árbol.
- Información de línea.

Entre los nodos implementados se encuentran:

- ProgramNode.
- FunctionNode.
- VarDeclNode.
- AssignmentNode.
- BinaryOpNode.
- IfNode.
- WhileNode.
- ReturnNode.
- FunctionCallNode.
- MoldNode.
- AttributeAccessNode.

El AST actúa como estructura central para el análisis semántico y la generación de código.

Análisis Semántico

El análisis semántico constituye una de las etapas centrales del compilador. Esta fase verifica la coherencia contextual del programa utilizando la información almacenada en el AST y las tablas de símbolos.

El sistema implementa:

Verificación de Tipos

Se valida la compatibilidad entre operandos y operadores, permitiendo detectar:

Operaciones inválidas entre tipos.

Asignaciones incompatibles.

Retornos incorrectos.

Comparaciones no válidas.

El compilador soporta inferencia y propagación de tipos básicos para expresiones compuestas.

Validación de Declaraciones

El analizador semántico verifica que:

Toda variable haya sido declarada previamente.

No existan redeclaraciones inválidas.

Los identificadores pertenezcan al ámbito correspondiente.

Las funciones existan antes de ser invocadas.

Manejo de Ámbitos (Scopes)

El sistema implementa ámbitos anidados mediante pilas de contextos.

Cada bloque de código crea un nuevo entorno semántico que permite:

Encapsular variables locales.

Resolver identificadores jerárquicamente.

Controlar visibilidad.

Gestionar parámetros de funciones.

Validación de Funciones

Las llamadas a funciones son verificadas comprobando:

- Existencia de la función.
- Número correcto de parámetros.
- Compatibilidad de tipos.
- Tipos de retorno.
- Validación de Estructuras

El sistema soporta validación semántica sobre moldes y acceso a atributos mediante el operador punto.

Durante esta etapa se verifica:

- Existencia del molde.
- Existencia del atributo.
- Compatibilidad de acceso.
- Uso correcto de estructuras.
- Tabla de Símbolos

La tabla de símbolos almacena información detallada de:

- Variables.
- Funciones.
- Parámetros.
- Moldes.
- Tipos.
- Alcance.
- Línea de declaración.
- Categoría semántica.

Esta estructura es utilizada durante todo el proceso de compilación.

Generación de Código

La generación de código transforma el AST validado semánticamente en instrucciones ensamblador compatibles con la arquitectura CacaoCore-64.

Durante esta etapa:

Las expresiones se traducen a operaciones ASM.
Las variables son asignadas a posiciones de memoria.
Las funciones generan etiquetas y manejo de stack.
Los condicionales producen saltos y etiquetas.
Los ciclos generan estructuras de control iterativas.

La generación sigue un modelo imperativo basado en:

Registros temporales.
Manejo explícito de stack.
Etiquetas de salto.
Traducción secuencial.

El código producido es posteriormente procesado por el ensamblador.

Ensamblador

El módulo ensamblador fue diseñado bajo un enfoque modular y orientado a objetos, separando claramente las responsabilidades de análisis léxico, análisis sintáctico y generación de código. Su objetivo principal es traducir programas escritos en lenguaje ensamblador a una representación en código máquina compatible con la arquitectura CacaoCore-64.

La arquitectura del ensamblador se compone de tres clases principales:

AsmLexer: encargado del análisis léxico.
AsmParser: responsable del análisis sintáctico y la traducción.
Assembler: punto de entrada que coordina el proceso completo.
Análisis Léxico

El análisis léxico se implementa mediante la clase AsmLexer, utilizando la herramienta PLY. Este componente define el conjunto de tokens reconocidos por el lenguaje ensamblador, incluyendo:

Mnemónicos (MNEMONIC): instrucciones del ISA como add, mov, jmp, etc.
Registros (REGISTER): identificadores de registros (r0 a r15).
Números (NUMBER): constantes enteras.
Flotantes (FLOAT): constantes en punto flotante.
Direcciones de memoria (MEMORY): valores hexadecimales (0x...).

Variables (VAR) y etiquetas (LABEL): identificadores simbólicos.

Secciones (SECTION): .data y .text.

Símbolos estructurales: :, ,, [,], saltos de línea.

El lexer distingue entre identificadores y mnemónicos mediante una verificación contra un conjunto predefinido de instrucciones. Además, ignora comentarios y espacios en blanco, permitiendo un procesamiento limpio del código fuente.

Análisis Sintáctico y Traducción

El análisis sintáctico se implementa en la clase AsmParser, la cual define una gramática basada en reglas de producción utilizando PLY (yacc). Esta gramática permite reconocer:

Definición de etiquetas

Declaración de variables en la sección .data

Instrucciones en la sección .text

Diferentes modos de direccionamiento

El parser mantiene varias estructuras internas clave:

Tabla de símbolos (symbol_table): almacena etiquetas y sus direcciones.

Tabla de variables (var_table): asigna direcciones a variables.

Lista de instrucciones (program_text) y datos (program_data)**.

Lista de referencias pendientes (pending) para etiquetas no resueltas.

Las instrucciones son traducidas directamente a código hexadecimal utilizando la especificación del ISA (MICROINSTRUCTION_SPECS). Dependiendo del tipo de instrucción, se manejan distintos formatos, tales como:

Registro–Registro

Registro–Inmediato

Registro–Memoria

Memoria–Registro

Acceso indirecto

Saltos con etiquetas

Durante la traducción, se realizan conversiones explícitas a formato binario utilizando codificación en little-endian, y se construyen las instrucciones como cadenas hexadecimales listas para ser cargadas en memoria.

Para el manejo de etiquetas, el ensamblador soporta referencias adelantadas mediante una lista de pendientes, que posteriormente se resuelven una vez finalizado el análisis completo del programa.

Generación de Salida

La clase `Assembler` actúa como punto de entrada al módulo. Esta encapsula el uso del parser y provee métodos para:

- Procesar código ensamblador de entrada.
- Resolver referencias pendientes.
- Generar la salida final en formato texto.
- Exportar el resultado a un archivo.

El resultado final consiste en una secuencia de instrucciones en formato hexadecimal, organizada según las secciones `.data` y `.text`, lista para ser cargada en memoria del emulador.

Enlazador-cargador

El módulo enlazador-cargador fue diseñado bajo un enfoque modular y orientado a objetos, separando claramente las responsabilidades de análisis léxico de módulos objeto, resolución de símbolos, validación de memoria y carga en el emulador. Su objetivo principal es tomar el código relocizable generado por el ensamblador, resolver referencias entre múltiples módulos objeto, y cargar el binario ejecutable resultante en la memoria del procesador CacaoCore-64.

La arquitectura del enlazador-cargador se compone de cuatro clases principales:

- `AnalizadorModulos`: encargado del análisis léxico de módulos objeto.
- `EnlazadorMejorado`: responsable del enlazado y resolución de símbolos.
- `CargadorMejorado`: encargado de la validación y carga en memoria.
- `GestorEnlazadorCargador`: punto de entrada que coordina el proceso completo.

Análisis Léxico de Módulos Objeto:

El análisis léxico se implementa mediante la clase `AnalizadorModulos`, utilizando la herramienta `PLY`. Este componente define el conjunto de tokens reconocidos por el lenguaje de módulos objeto, incluyendo:

Palabras clave (SECTION): [MODULE, CODE, DATA, SYMBOLS, EXTERNAL]
Identificadores (ID): [a-zA-Z][a-zA-Z0-9_]*
Números hexadecimales (HEX): 0x[0-9A-Fa-f]+ o [0-9A-Fa-f]+
Números decimales (NUM): [0-9]+
Delimitadores: :, ,, [,], {, }
Símbolos especiales: whitespace, saltos de línea

El lexer tokeniza la estructura de módulos objeto que tiene el formato:

[MODULE nombre_modulo]

[CODE] HEX HEX HEX HEX ...

[DATA] HEX HEX HEX HEX ...

[SYMBOLS] simbolo1:tipo:valor, simbolo2:tipo:valor, ...

[EXTERNAL] ref1:posicion:tipo, ref2:posicion:tipo, ...

Donde:

- nombre_modulo es un identificador único del módulo
- HEX son bytes en formato hexadecimal (00-FF)
- tipo puede ser "code" o "data"
- valor es la dirección relativa al módulo (en hex con prefijo 0x o decimal)
- posicion es el offset dentro del módulo donde resolver la referencia
- ref es el nombre del símbolo externo a resolver

Análisis Sintáctico y Enlazado

El proceso de enlazado se implementa en la clase `EnlazadorMejorado`, la cual coordina la resolución de referencias entre múltiples módulos objeto. Este componente mantiene varias estructuras internas clave:

Tabla de símbolos global (`symbol_table`): almacena todos los símbolos resueltos y sus direcciones finales.

Tabla de referencias pendientes (`pending_references`): mantiene referencias no resueltas para resolución posterior.

Gestor de módulos (`modules`): colección de módulos parseados con `AnalizadorModulos`.

Tabla de símbolos por módulo: mapeo de identificadores a direcciones relativas dentro de cada módulo.

El proceso de enlazado realiza las siguientes operaciones:

1. Carga de Módulos: Cada archivo de módulo objeto se parsea utilizando `AnalizadorModulos`, generando una estructura `ParsedModule` que contiene el código, datos y tabla de símbolos del módulo.

2. Construcción de Tabla de Símbolos Global:

- Iterar sobre todos los módulos
- Para cada símbolo interno, ajustar su valor según la dirección base asignada al módulo
- Detectar conflictos (símbolos duplicados)
- Almacenar en tabla global con dirección final resuelta

3. Resolución de Referencias Externas:

- Por cada referencia pendiente en cada módulo:
- Buscar el símbolo en la tabla global
- Si existe: calcular la dirección final y actualizar la instrucción en el módulo
- Si no existe: registrar error de símbolo no resuelto

4. Construcción de Binario Ejecutable:

- Concatenar todas las secciones CODE en orden de módulos

- Concatenar todas las secciones DATA en orden de módulos
- Incluir tabla de símbolos global
- Crear estructura BinarioEjectable con metadata

Validación y Carga en Memoria

La clase CargadorMejorado encapsula la lógica de validación de memoria e inicialización del procesador. Este componente:

Valida que la dirección base sea válida ($\geq 0x1000$ para programas de usuario) Verifica que el código quepa en la zona CODE (0x1000-0x3FFFF, 252 KB) Verifica que los datos quepan en la zona DATA (0x40000-0x7FFFF, 256 KB) Valida que no existan solapamientos entre secciones Comprueba disponibilidad de RAM en el emulador

Una vez validado el binario, el cargador:

1. Detiene el procesador temporalmente (si está activo)
2. Limpia registros temporales (MAR, MDR) para evitar conflictos
3. Escribe el código en memoria a partir de la dirección base
4. Escribe los datos en la zona DATA designada
5. Inicializa registros del procesador:
 - PC (Program Counter): dirección base
 - SP (Stack Pointer): tope de la pila (STACK_END)
 - Flags de estado: todos en cero
 - Registros acumuladores: limpios
6. Reactiva el procesador y retorna estado de éxito

Generación de Salida

La clase `GestorEnlazadorCargador` actúa como punto de entrada al módulo, proporcionando una interfaz simplificada que encapsula el uso del enlazador y cargador. Este componente ofrece métodos para:

Procesar múltiples archivos de módulos objeto desde disco
Procesar módulos desde contenido de texto directo
Resolver referencias y generar binario ejecutable
Cargar el binario en memoria del emulador
Recuperar tabla de símbolos global
Obtener descripciones de errores ocurridos.

El resultado final consiste en código y datos residentes en memoria del CacaoCore-64, con el procesador inicializado y listo para comenzar ejecución desde la dirección especificada. El sistema soporta múltiples módulos objeto con resolución automática de símbolos entre ellos, permitiendo compilación modular de programas complejos.

5. Código Fuente

El código fuente se encuentra adjunto en el entregable T2 Correa Escobar Sergio Nicolás 02.zip

6. Manual de Usuario

CacaoCore-64 es un emulador educativo de un procesador de 64 bits que implementa una arquitectura Von Neumann con un espacio de memoria RAM de 1 MB y un conjunto de instrucciones personalizado. La herramienta permite comprender el funcionamiento interno de un procesador mediante la visualización y manipulación directa de registros, memoria y flujo de ejecución. El presente manual describe los requisitos, el procedimiento de ejecución, el uso de la interfaz gráfica y las pautas básicas de operación y resolución de problemas.

a. Requisitos

Para ejecutar el emulador se requiere:

- Python 3.8 o superior.
- Tkinter (incluido por defecto en la mayoría de instalaciones de Python).
- PLY (librería que incluye los equivalentes de LEX y YACC en Python). Opcionalmente, se puede utilizar uv sync
- Aproximadamente 10 MB de espacio en disco.
- Resolución recomendada: 1920x1080.

Para verificar Python, se puede usar el comando “python --version”.

b. Ejecución e Interfaz Principal

Desde la carpeta raíz del proyecto, después de descomprimir el archivo .zip:

“python -m src.cacao_gui”.

La interfaz se abrirá automáticamente en pantalla, para salir, cierre la ventana.

La interfaz principal constituye el núcleo de interacción con el procesador simulado. En ella se visualizan los registros fundamentales del sistema, los controles de ejecución y las opciones de visualización de datos.

En la sección superior se muestran los registros de control más relevantes: el Program Counter (PC), que indica la dirección de la siguiente instrucción a ejecutar; el Instruction Register (IR), que contiene la instrucción actualmente en proceso; y el Flags Register (FR), que almacena indicadores de estado generados por operaciones aritméticas y lógicas. Estos indicadores incluyen las banderas de interrupción (I), desbordamiento (V),

acarreo (C), negativo (N) y cero (Z), las cuales permiten evaluar condiciones posteriores a una operación.

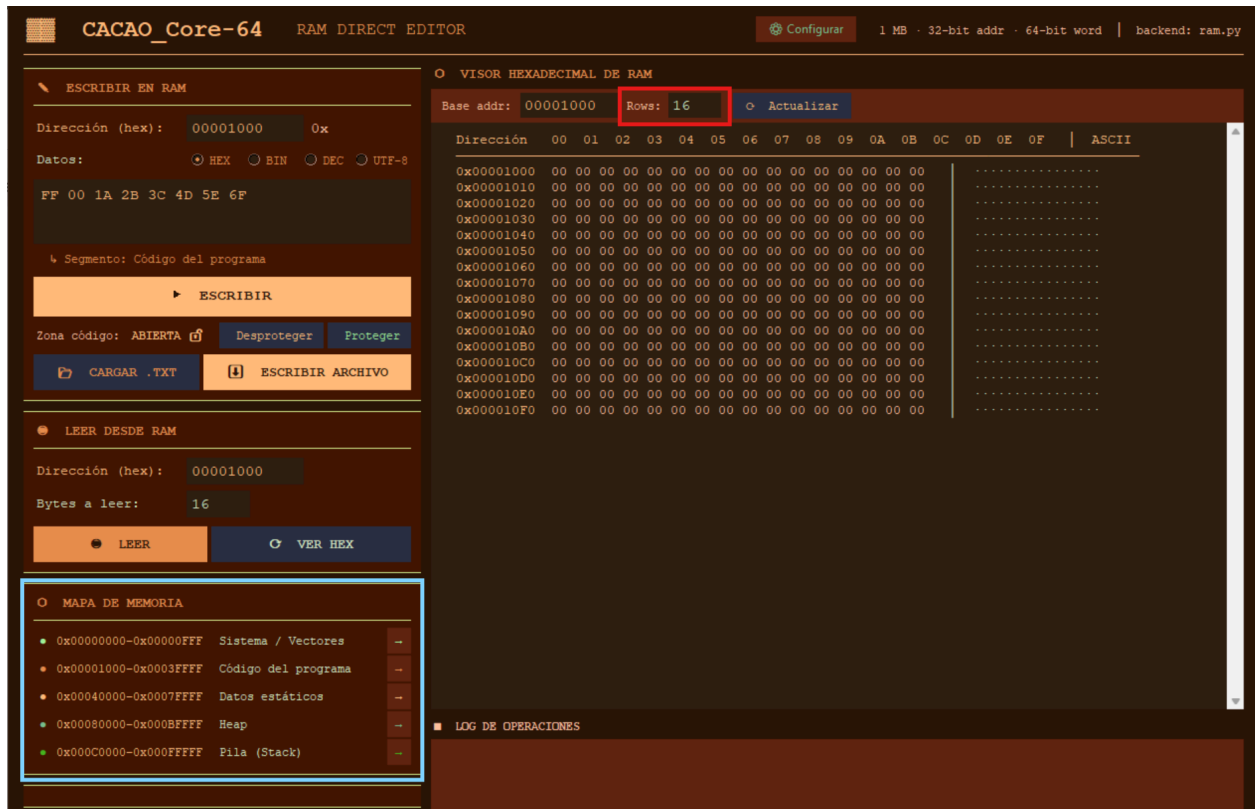
Debajo de estos registros se encuentran los 16 registros de propósito general de 64 bits. Los registros r0 a r12 pueden emplearse libremente para almacenamiento temporal de datos. El registro r13 corresponde al Stack Pointer (\$sp), que apunta al tope de la pila; r14 es el Link Register (\$lr), utilizado para almacenar direcciones de retorno en llamadas a funciones; y r15 es el Acumulador (\$acc), empleado por la unidad aritmético-lógica para almacenar resultados.

La interfaz incorpora tres controles principales de ejecución. El botón BOOT reinicia completamente el procesador, limpia los registros y posiciona el Program Counter en la dirección inicial especificada por el usuario. El botón RUN FULL ejecuta el programa de manera continua hasta encontrar una instrucción HLT o producirse un error. El botón RUN STEP permite ejecutar una única instrucción por vez, facilitando la depuración detallada del código.

El usuario debe especificar la dirección de inicio en formato hexadecimal, sin prefijo “0x”. Asimismo, puede seleccionar el formato de visualización de los registros entre hexadecimal, decimal o binario. Esta selección únicamente afecta la representación visual de los datos y no altera su valor interno.

c. Gestión y Edición de Memoria RAM

El emulador dispone de un editor de memoria RAM accesible desde la interfaz principal. La memoria total es de 1 MB y se encuentra segmentada lógicamente en zonas destinadas a sistema, código, datos estáticos, heap y stack. Puede hacer uso de la sección **mapa de memoria** para poder explorar las distintas zonas de la ram y así mismo puede modificar el número de filas a visualizar cambiando el valor de **rows**:

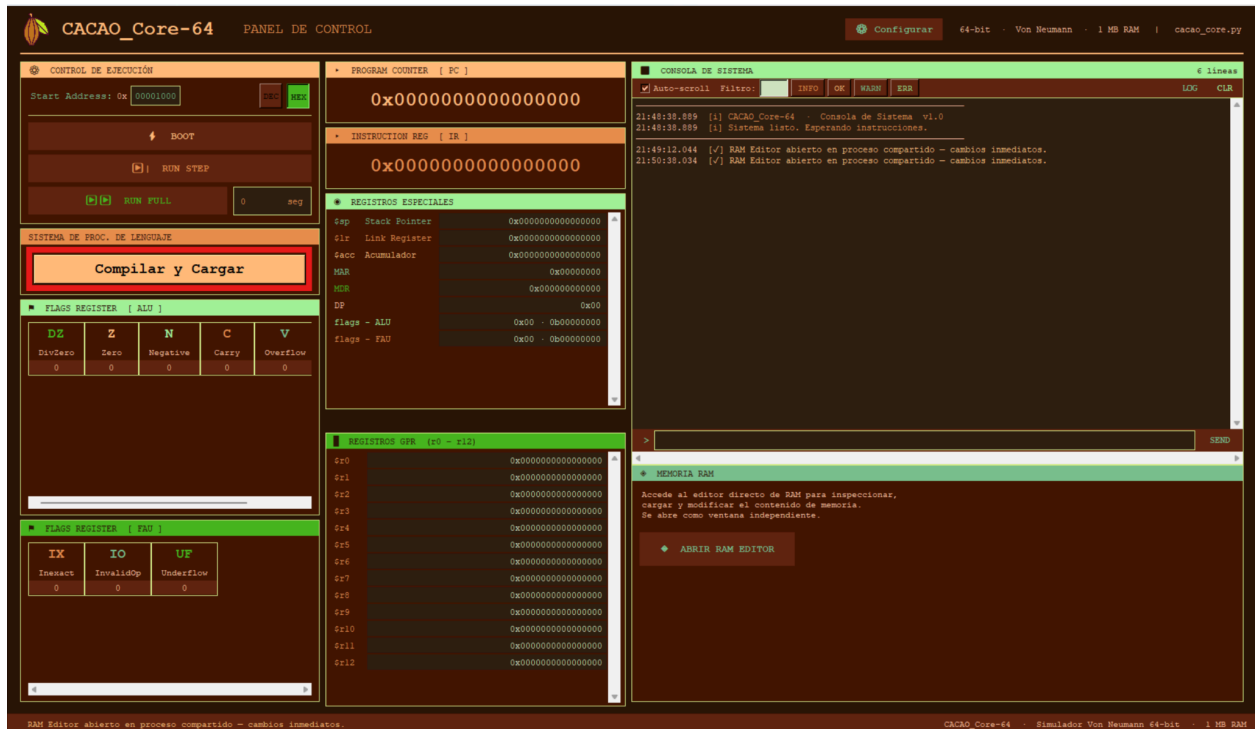


La zona de código comienza en la dirección hexadecimal 0x00001000, siendo el punto habitual para cargar programas. El usuario puede introducir los bytes manualmente, cargarlos desde un archivo en formato binario o hexadecimal, o modificarlos directamente por dirección. Cada celda representa un byte expresado en dos dígitos hexadecimales.

Una vez cargado el programa, la información permanece en memoria incluso después de cerrar el editor, permitiendo regresar a la interfaz principal para iniciar la ejecución.

d. Procesamiento de Lenguaje

Para acceder a esta presione el botón “Compilar y Cargar” que se muestra a continuación:



Posterior al click se debe de abrir la siguiente ventana auxiliar automáticamente:



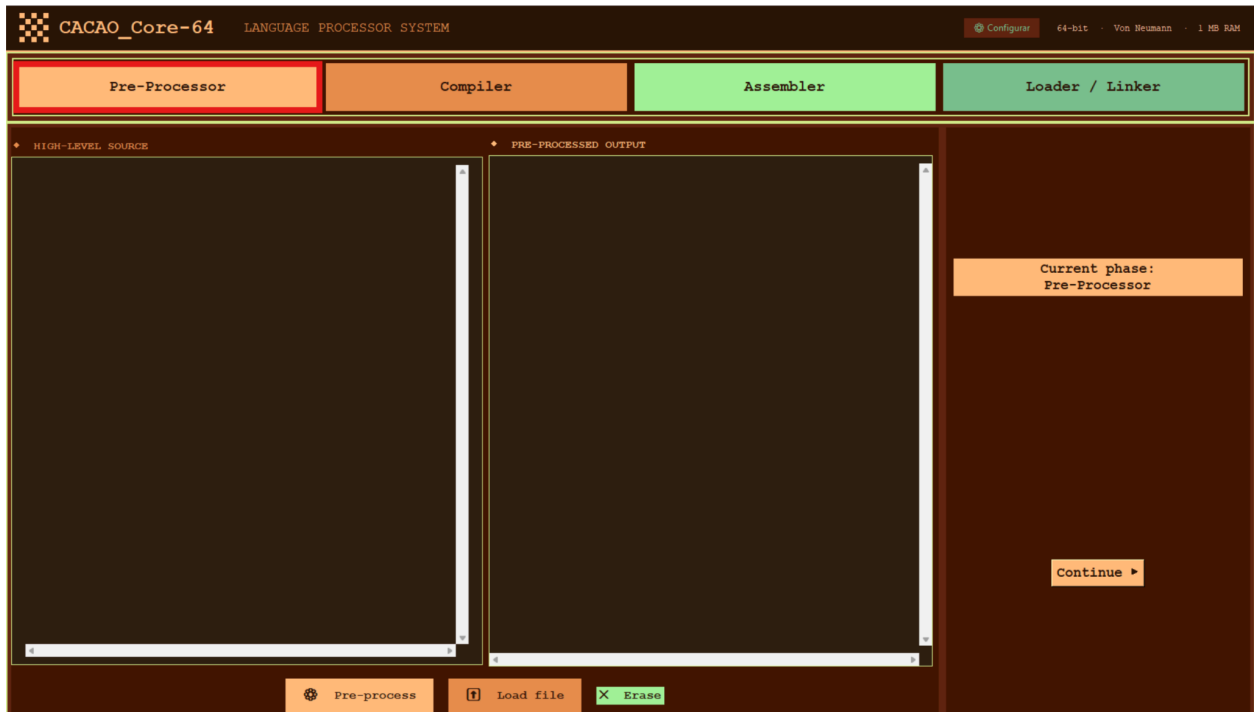
Esta cuenta con las fases propuestas para elaborar para esta tarea, que corresponden a:

- Pre-procesador
- Compiler → Analisis Lexico → Analisis Sintactico → Analisis Semantico
- Assembler

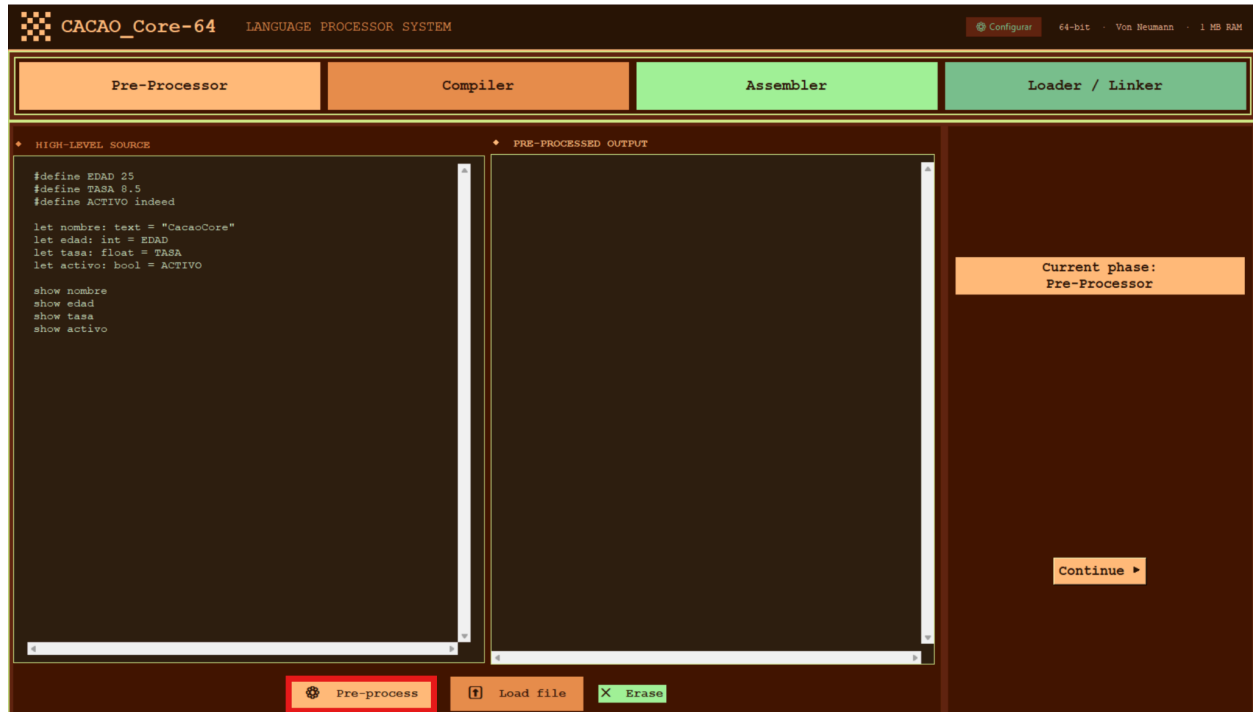
- Linker y Loader

A continuación abordaremos el funcionamiento de cada una de estas fases:

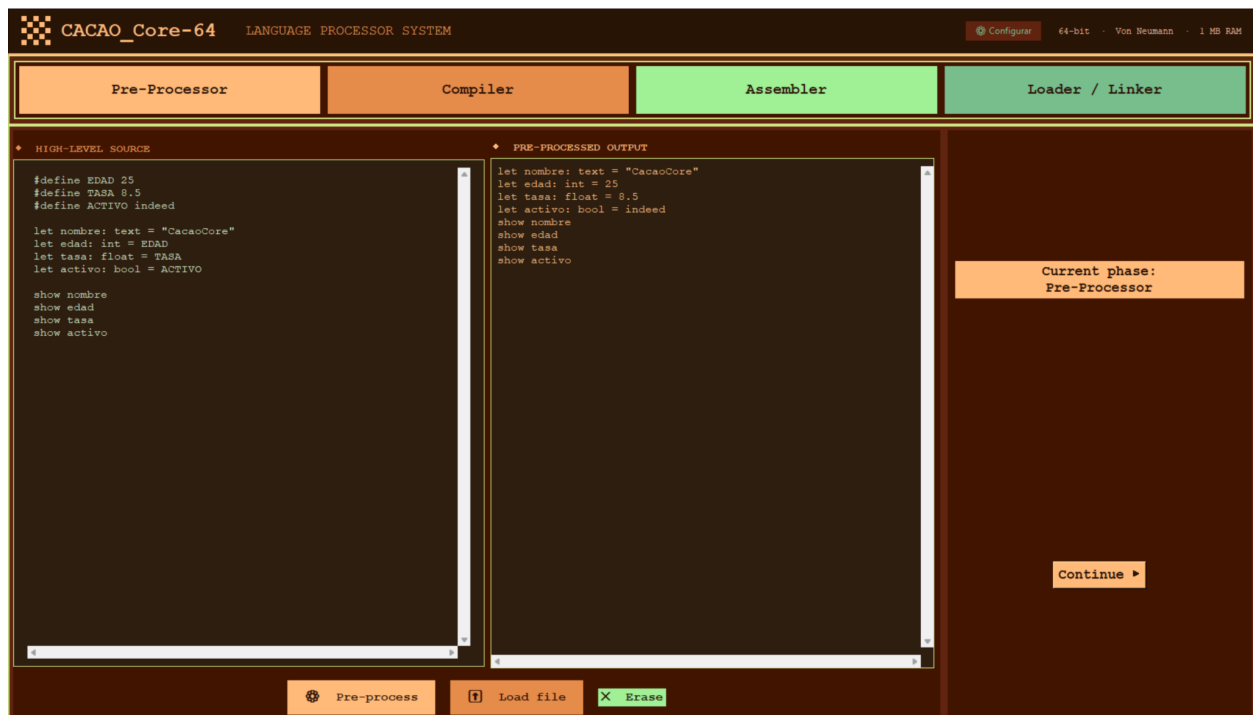
1. Pre-procesador:



Es posible acceder a esta fase presionando su respectivo botón resaltado con rojo. Para comenzar está la posibilidad de subir un archivo presionando el botón load file y escogiendo el archivo que sea de interés o si se desea escribir directamente el código en el campo:



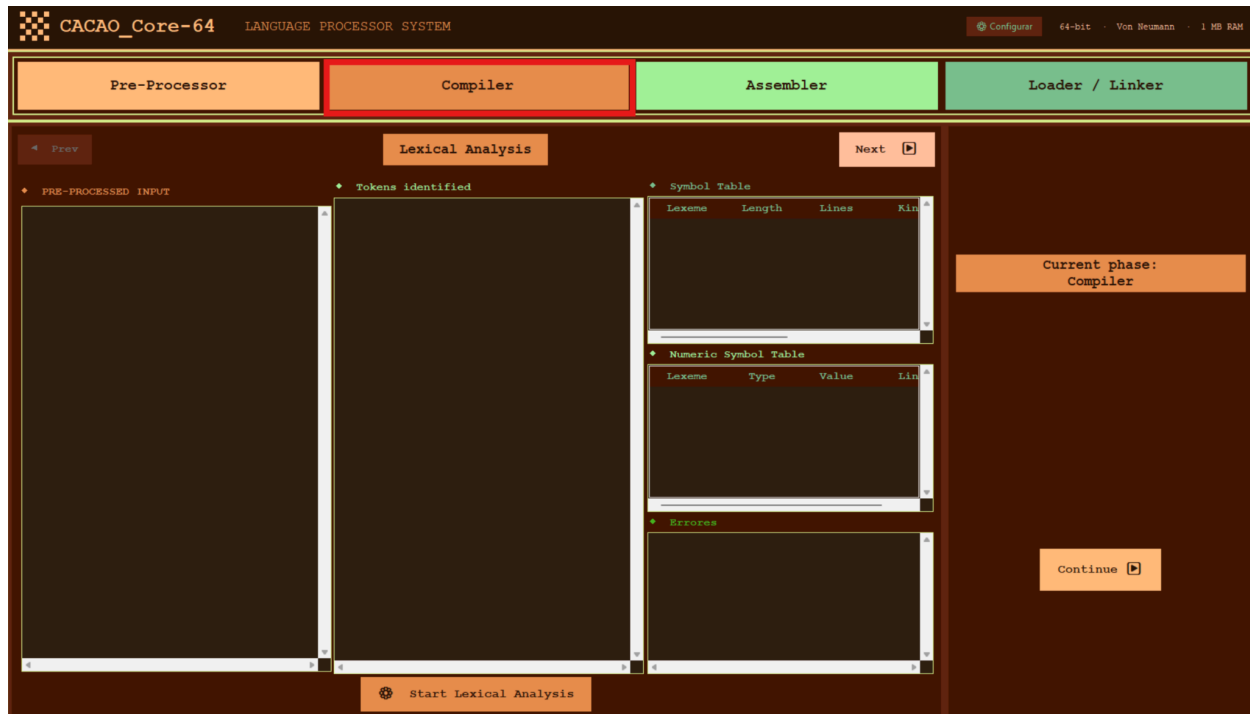
Ya con el código a procesar en el campo izquierdo se presiona el botón encerrado en la imagen para así ver la correspondiente salida pre-procesada:



Esta salida está lista para pasar por el proceso de compilación, para acceder a la siguiente fase del procesamiento presione el botón “Continue” ubicado a la derecha de la interfaz.

2.

a. Análisis Léxico:



Esta fase es accedida ya sea mediante el botón continue desde la sección pre-processor o directamente mediante el botón encerrado “Compiler”, el campo de más a la izquierda contendrá el código fuente a compilar que podrá ser escrito directamente o puede ser producto de la fase de preprocesamiento, para empezar el analisis lexico presione con el campo input ya cargado el botón “Start Lexical Analysis”:

CACAO_Core-64 LANGUAGE PROCESSOR SYSTEM

Configurar 64-bit Von Neumann 1 MB RAM

Pre-Processor Compiler Assembler Loader / Linker

Lexical Analysis

Prev Next

PRE-PROCESSED INPUT

```
let a: int = 756
let b: int = 924
asLongAs a != b {
  if a > b {
    set a = a - b
  } otherwise {
    set b = b - a
  }
}
show a
```

Tokens identified

```
LET: let
ID: a
COLON: :
INT_TYPE: int
ASSIGN: =
INT_LIT: 756
LET: let
ID: b
COLON: :
INT_TYPE: int
ASSIGN: =
INT_LIT: 924
ASLONGAS: asLongAs
ID: a
NEQ: !=
ID: b
LBRACE: {
IF: if
ID: a
GT: >
ID: b
LBRACE: {
SET: set
ID: a
ASSIGN: =
ID: a
MINUS: -
ID: b
LBRACE: }
OTHERWISE: otherwise
LBRACE: {
SET: set
ID: b
LIT: -
```

Symbol Table

Lexeme	Length	Lines	Kind
a	1	1 3 4 5 5	Non
b	1	2 3 4 5 7	Non

Numeric Symbol Table

Lexeme	Type	Value	Lin
756	int	756	1
924	int	924	2

Errores

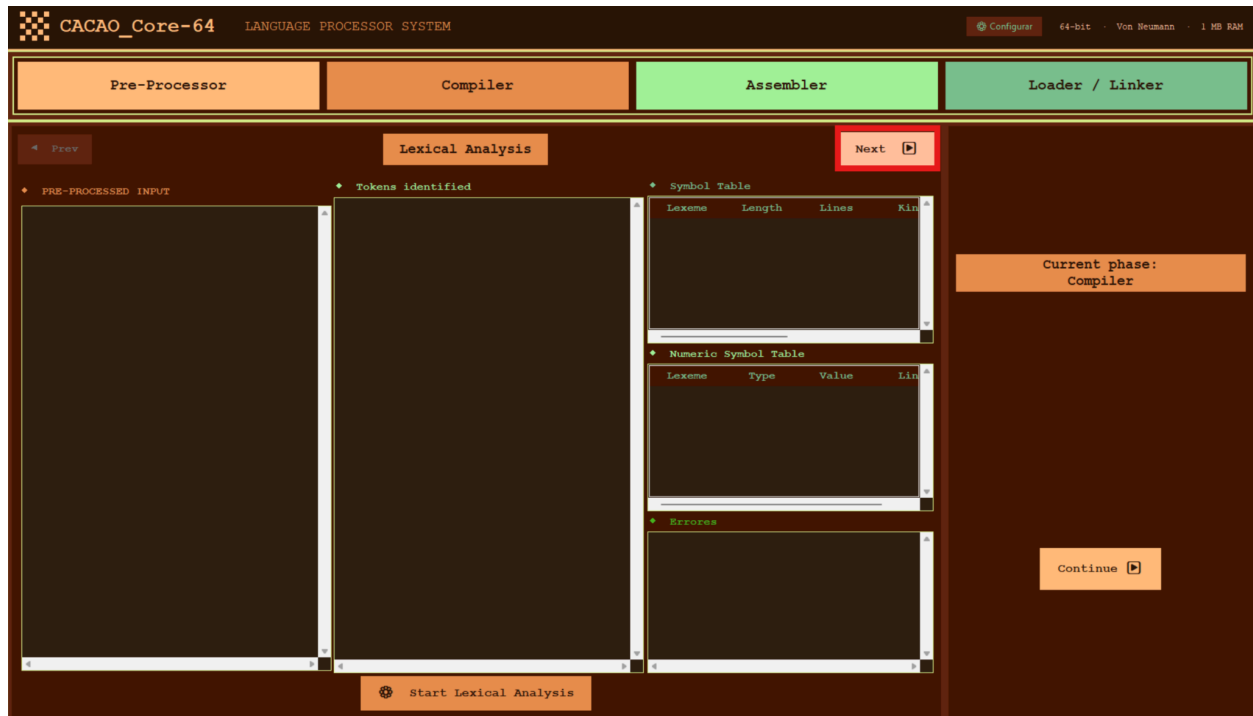
Current phase: Compiler

Continue

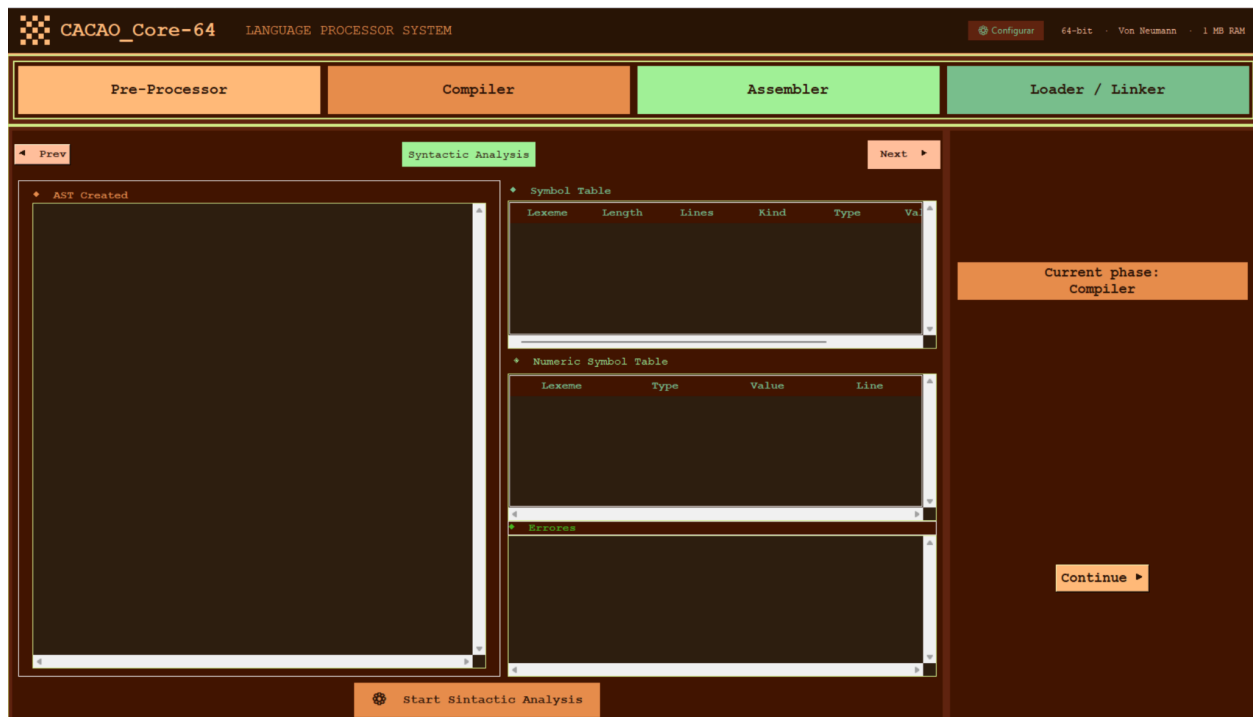
Start Lexical Analysis

El resultado de este análisis léxico es todos los tokens encontrados en el campo tokens identified, la tabla de símbolos y de números actualizada con los lexemas su longitud y las líneas donde aparecen y una ventana de error que informa de tokens que no pudieron ser reconocidos.

Si desea continuar con la siguiente fase del procesamiento de lenguaje presione el botón next encima de la tabla de símbolos:



b. Análisis Sintáctico:



Para ejecutar esta fase presione el botón “Start Syntactic Analysis”, una vez presionado la salida será el árbol AST correspondiente al código proporcionado:

The screenshot shows the CACAO_Core-64 LANGUAGE PROCESSOR SYSTEM interface. The top navigation bar includes 'Pre-Processor', 'Compiler', 'Assembler', and 'Loader / Linker'. The 'Compiler' phase is active, and the 'Syntactic Analysis' sub-phase is selected. The interface displays the following components:

- AST Created:** A tree structure showing the parsed code. The root is 'program', which contains a list of sentences. The first sentence is a 'let_stmt' with variables 'a' and 'b' assigned the value 756. The second sentence is a 'while_stmt' with a condition 'a == b' and a body containing an 'if_stmt'.
- Symbol Table:** A table with columns: Lexeme, Length, Lines, Kind, Type, Value. It contains two entries: 'a' (length 1, lines 1 3 4 5 5, kind None, type None, value None) and 'b' (length 1, lines 2 3 4 5 7, kind None, type None, value None).
- Numeric Symbol Table:** A table with columns: Lexeme, Type, Value, Line. It contains two entries: '756' (type int, value 756, line 1) and '924' (type int, value 924, line 2).
- Errors:** A table with no visible entries.
- Buttons:** 'Prev', 'Next', 'Start Sintactic Analysis', and 'Continue'.
- Current phase:** Compiler.

Para avanzar a la siguiente etapa presione nuevamente el botón next.

c. Análisis Semántico y Generación de código

The screenshot shows the CACAO_Core-64 LANGUAGE PROCESSOR SYSTEM interface. The top navigation bar includes 'Pre-Processor', 'Compiler', 'Assembler', and 'Loader / Linker'. The 'Compiler' phase is active, and the 'Semantic Analysis' sub-phase is selected. The interface displays the following components:

- ANNOTATED AST:** A tree structure showing the annotated code. The root is 'program', which contains a list of sentences. The first sentence is a 'let_stmt' with variables 'a' and 'b' assigned the value 756. The second sentence is a 'while_stmt' with a condition 'a == b' and a body containing an 'if_stmt'.
- Symbol Table:** A table with columns: Lexeme, Length, Lines, Kind, Type, Value. It contains two entries: 'a' (length 1, lines 1 3 4 5 5, kind None, type None, value None) and 'b' (length 1, lines 2 3 4 5 7, kind None, type None, value None).
- Numeric Symbol Table:** A table with columns: Lexeme, Type, Value, Line. It contains two entries: '756' (type int, value 756, line 1) and '924' (type int, value 924, line 2).
- Errors:** A table with no visible entries.
- Buttons:** 'Prev', 'Next', 'Start Semantic Analysis', 'Generate code', and 'Continue'.
- Current phase:** Compiler.

Esta sección cuenta con dos botones, el primero resaltado en rojo procederá con el análisis semántico y tendrá como salida el árbol ast anotado, por el otro lado, el

segundo botón (que ha de presionar luego del primero) tendrá como salida el ASM a procesar por el siguiente módulo:

CACAO_Core-64 LANGUAGE PROCESSOR SYSTEM Configur 64-bit Von Neumann 1 MB RAM

Pre-Processor Compiler **Assembler** Loader / Linker

Prev Semantic Analysis Next

ANNOTATED AST

```

program
  sentencias: list[4]
  [0]: let_stmt
    let_keyword: "let"
    nombre: 'a'
    colon: ":"
    tipo: "int"
    dims: 0
    assign: "="
    valor: INT_LIT
    valor: 756
    tipo: 'int'
    dims: 0
    kind: 'variable'
    scope_id: 'global@1'
  [1]: let_stmt
    let_keyword: "let"
    nombre: 'b'
    colon: ":"
    tipo: "int"
    dims: 0
    assign: "="
    valor: INT_LIT
    valor: 924
    tipo: 'int'
    dims: 0
    kind: 'variable'
    scope_id: 'global@2'
  [2]: while_stmt
    aslongas_keyword: "asLongAs"
    condicion: expr_binop
    isq: ID
    nombre: 'a'
    tipo: 'int'
    dims: 0
    kind: 'variable'
  
```

Symbol Table

Lexeme	Length	Lines	Kind
a	1	1 3 4 5 5	variable
b	1	2 3 4 5 7	variable

Numeric Symbol Table

Lexeme	Type	Value	Line
756	int	756	1
924	int	924	2

Generated code

```

.data
  a: 756
  b: 924
.text
  LWHILE_START3:
    MOV R0, [a]
    CMP R0, [b]
    JNE LTRUE0
    MOV R0, 0
    JMP LFALSE0
  LTRUE0:
    MOV R0, 1
    LFALSE0:
    CMP R0
    JG LWHILE_END3
    MOV R2, [a]
    CMP R2, [b]
    JG LTRUE1
    MOV R2, 0
    JMP LFALSE1
  LTRUE1:
    MOV R2, 1
    LFALSE1:
    CMP R2
    JG LFALSE2
    MOV R3, [a]
    SUB R3, [b]
    MOV [a], R3
    MOV R3, [a]
    MOV [b], R3
    LEND1:
    JMP LWHILE_START3
  LWHILE_END3:
  
```

Current phase: Compiler

Continue

Start Semantic Analysis Generate code

CACAO_Core-64 LANGUAGE PROCESSOR SYSTEM Configur 64-bit Von Neumann 1 MB RAM

Pre-Processor Compiler **Assembler** Loader / Linker

Prev Semantic Analysis Next

ANNOTATED AST

```

program
  sentencias: list[4]
  [0]: let_stmt
    let_keyword: "let"
    nombre: 'a'
    colon: ":"
    tipo: "int"
    dims: 0
    assign: "="
    valor: INT_LIT
    valor: 756
    tipo: 'int'
    dims: 0
    kind: 'variable'
    scope_id: 'global@1'
  [1]: let_stmt
    let_keyword: "let"
    nombre: 'b'
    colon: ":"
    tipo: "int"
    dims: 0
    assign: "="
    valor: INT_LIT
    valor: 924
    tipo: 'int'
    dims: 0
    kind: 'variable'
    scope_id: 'global@2'
  [2]: while_stmt
    aslongas_keyword: "asLongAs"
    condicion: expr_binop
    isq: ID
    nombre: 'a'
    tipo: 'int'
    dims: 0
    kind: 'variable'
  
```

Symbol Table

Lexeme	Length	Lines	Kind
a	1	1 3 4 5 5	variable
b	1	2 3 4 5 7	variable

Numeric Symbol Table

Lexeme	Type	Value	Line
756	int	756	1
924	int	924	2

Generated code

```

.data
  a: 756
  b: 924
.text
  LWHILE_START3:
    MOV R0, [a]
    CMP R0, [b]
    JNE LTRUE0
    MOV R0, 0
    JMP LFALSE0
  LTRUE0:
    MOV R0, 1
    LFALSE0:
    CMP R0
    JG LWHILE_END3
    MOV R2, [a]
    CMP R2, [b]
    JG LTRUE1
    MOV R2, 0
    JMP LFALSE1
  LTRUE1:
    MOV R2, 1
    LFALSE1:
    CMP R2
    JG LFALSE2
    MOV R3, [a]
    SUB R3, [b]
    MOV [a], R3
    MOV R3, [a]
    MOV [b], R3
    LEND1:
    JMP LWHILE_START3
  LWHILE_END3:
  
```

Current phase: Compiler

Continue

Start Semantic Analysis Generate code

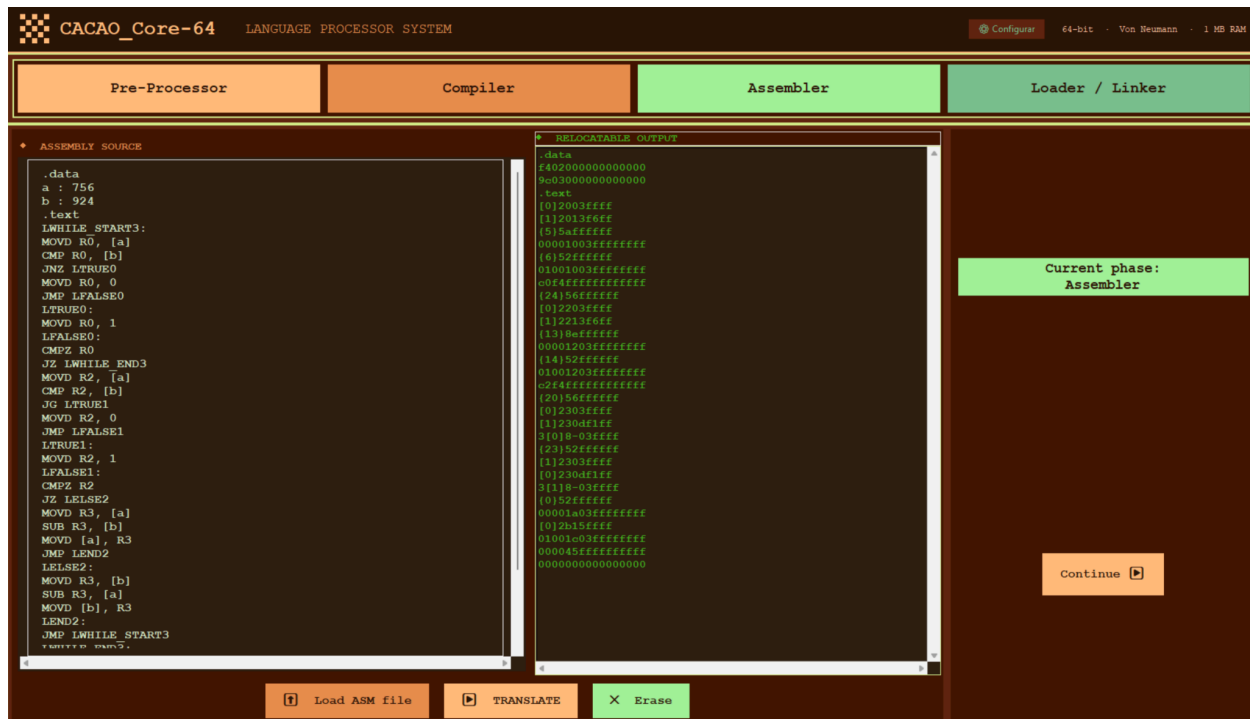
Una vez terminado el flujo presione el botón continue para pasar al módulo ensamblador.

3. Assembler:

Esta fase puede también ser accedida mediante su botón correspondiente “Assembler”:



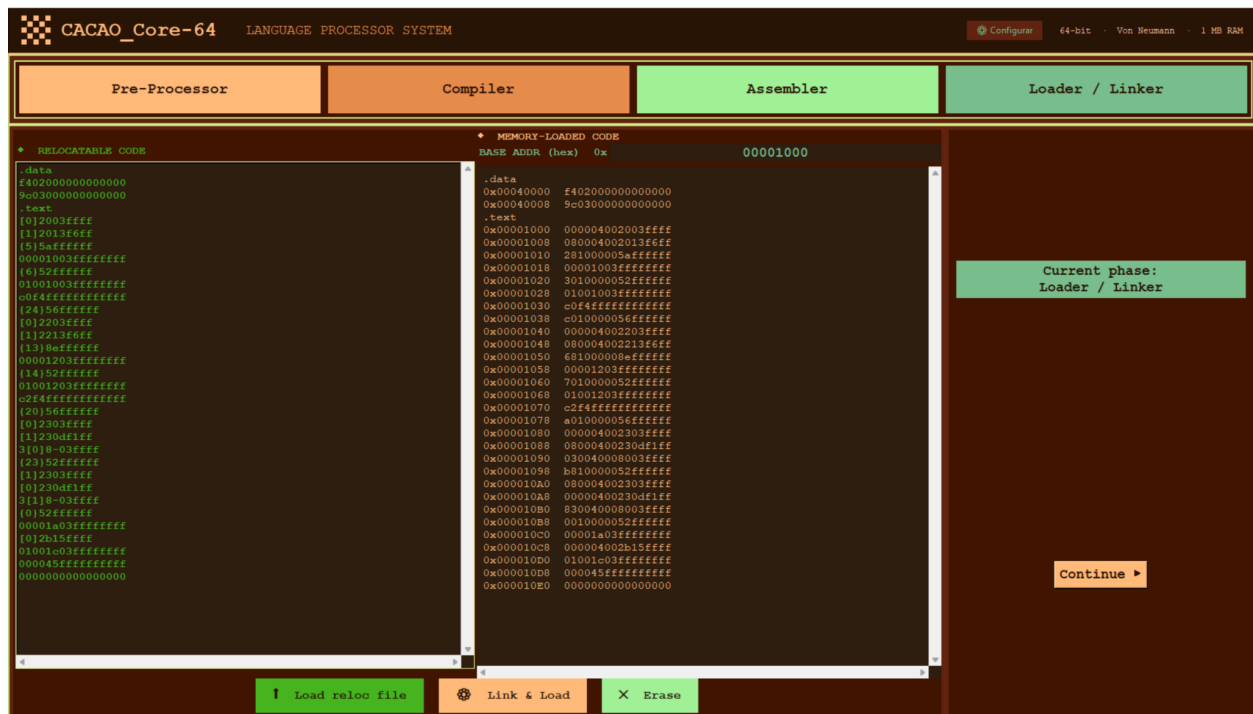
En la interfaz habrá dos campos uno con el código en asm y otro que recibirá el output de código relocable correspondiente. Si desea subir un archivo a él source haga uso del botón “Load ASM file” o escriba el código asm directamente:



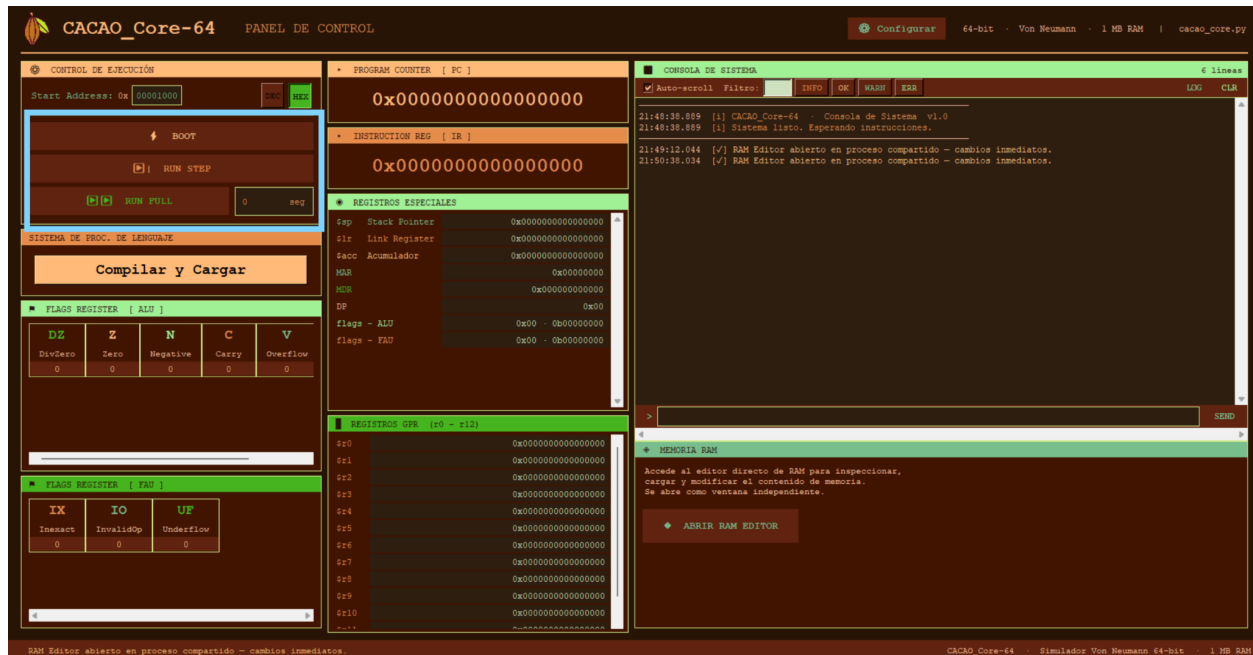
Con el código a traducir en el campo izq presione el botón “TRANSLATE” con el cual producirá el código reloc resultante como se muestra en la imagen previa, para continuar con el flujo del procesamiento presione continue una vez finalizada la etapa.

4. Linker y Loader:

En esta última etapa dada una dirección de memoria escogida previamente en el entry de la sección se calcularán las direcciones absolutas para todas aquellas referencias relativas y posteriormente se escribirá todo el código binario resultante en la ram desde la debida posición de memoria.



Una vez completado el procesamiento del lenguaje el código se encontrará alocado en la memoria ram. Bootee el computador y haga la corrida del código ya sea mediante la opción paso a paso “run step” o la completa “run full”:



e. Consideraciones

Si el programa no avanza o parece detenido, es probable que se encuentre en un bucle infinito o que no contenga una instrucción HLT. En estos casos se recomienda reiniciar con BOOT y utilizar el modo RUN STEP para identificar el punto de bloqueo. Si el Program Counter no cambia tras ejecutar una instrucción, debe verificarse que el sistema haya sido reiniciado correctamente y que la dirección de inicio sea válida y esté en formato hexadecimal.

Cuando los registros presentan valores inesperados, la estrategia adecuada consiste en ejecutar el programa paso a paso para determinar qué instrucción genera la modificación no deseada. También es recomendable revisar las direcciones de memoria utilizadas. Los errores de tipo “Address Out of Range” indican que el programa intenta acceder a direcciones fuera del rango válido de la RAM (0x00000000 a 0x000FFFFF). Deben revisarse los operandos y cálculos de dirección.

Si el campo de dirección inicial no acepta la entrada, debe comprobarse que únicamente se utilicen caracteres hexadecimales válidos (0–9 y A–F) sin prefijo adicional. Finalmente, si la interfaz gráfica presenta problemas de visualización, se recomienda ajustar la resolución del sistema o revisar la configuración de escalado del sistema operativo.

CacaoCore-64 está diseñado como herramienta académica para el estudio de arquitectura de computadores. El uso del modo paso a paso, la observación sistemática de registros y

flags, y la verificación periódica de la memoria constituyen buenas prácticas para la comprensión profunda del comportamiento del procesador.

Este entorno permite experimentar de manera controlada con conceptos fundamentales como ejecución secuencial, manejo de pila, direccionamiento de memoria y control de flujo.

7. Manual Técnico

El presente documento describe con mayor profundidad el diseño e implementación de los componentes léxicos del SPL del emulador CacaoCore-64. Está dirigido a desarrolladores que requieran comprender el diseño del sistema, extender su funcionalidad o realizar procesos de depuración a nivel de código fuente y utilización avanzada.

a. Introducción y Visión General del Sistema

El presente documento describe de manera formal el análisis, diseño e implementación del Sistema de Procesamiento del Lenguaje SPL, desarrollado para la arquitectura CacaoCore-64 de 64 bits. Este sistema constituye una cadena completa de traducción que permite transformar programas escritos en el lenguaje SPL en código ejecutable, listo para ser cargado y ejecutado en memoria.

El sistema se ha concebido bajo un enfoque modular y multifase, en el cual cada componente cumple una función específica dentro del proceso de compilación y ejecución. En el contexto del Taller #01, se abordan cuatro componentes fundamentales: el preprocesador, el ensamblador, el enlazador-cargador y el módulo de análisis léxico del compilador. Cada uno de estos ha sido diseñado siguiendo principios clásicos de construcción de compiladores y empleando herramientas basadas en el metalenguaje FLEX, implementadas mediante PLY en Python.

El flujo general del sistema inicia con el código fuente SPL, pasa por una etapa de preprocesamiento donde se resuelven macros e inclusiones, continúa con el análisis léxico que transforma el código en una secuencia de tokens, y posteriormente atraviesa fases de análisis sintáctico y semántico. Finalmente, el código es traducido a ensamblador, convertido a código objeto por el ensamblador, enlazado con otros módulos y cargado en memoria para su ejecución por el procesador CacaoCore-64.

b. Preprocesador

El preprocesador constituye la primera etapa del sistema de traducción. Su propósito es preparar el código fuente antes de su análisis formal, permitiendo la expansión de macros y la inclusión de archivos externos. En este proyecto, el preprocesador ha sido diseñado e

implementado utilizando herramientas equivalentes al metalenguaje FLEX, específicamente mediante PLY, lo cual permite definir reglas léxicas de manera estructurada.

Desde el punto de vista del análisis, se identificó la necesidad de soportar al menos dos directivas fundamentales: `#include` y `#define`. Estas directivas son esenciales para la modularidad del código y la reutilización de fragmentos. El diseño del preprocesador contempla un mecanismo de lectura secuencial del código fuente, acompañado de una tabla de macros y un sistema de resolución de rutas para archivos incluidos.

En términos de implementación, cuando el preprocesador encuentra una directiva `#include`, no inserta el contenido del archivo en el flujo de entrada. En cambio, emite una directiva `.import` en la salida para que el enlazador sepa que ese módulo externo existe. Si se especifican funciones entre llaves, también emite una directiva `.extern` por cada una. El contenido del archivo referenciado nunca se expande en el código: es el enlazador quien, en una etapa posterior, une los módulos. Por ejemplo, `#include "lib.reloc" { suma }` genera: `.import "lib.reloc" y .extern suma` en la salida.

Un aspecto relevante del diseño es el mantenimiento de un mapeo de líneas de origen, lo cual permite asociar cada línea del código preprocesado con su ubicación original. Esto resulta fundamental para la generación de mensajes de error precisos en etapas posteriores del compilador, aunque aún resulta un poco difuso dada la incompletitud del compilador.

En conjunto, el preprocesador implementado cumple con los requerimientos establecidos, al gestionar correctamente macros de inclusión y definición, y proporcionar una salida coherente para las siguientes fases del sistema.

Además de `#include` y `#define`, el preprocesador soporta otras seis directivas que no se describen en los apartados anteriores:

- `#undef MACRO` — elimina una macro definida previamente. A partir de esa línea, la macro deja de existir.
- `#ifdef MACRO` — incluye el bloque siguiente solo si `MACRO` está definida.
- `#ifndef MACRO` — incluye el bloque siguiente solo si `MACRO` NO está definida.
- `#else` — define el bloque alternativo dentro de un `#ifdef` o `#ifndef`.
- `#endif` — cierra el bloque condicional `#ifdef` / `#ifndef`.
- `#error "mensaje"` — detiene el preprocesamiento y reporta un error inmediatamente.
- `#warning "mensaje"` — emite una advertencia sin detener el proceso.

Además, `#define` no solo define constantes simples. También soporta macros con parámetros que se expanden como si fueran funciones en tiempo de preprocesamiento.

Por ejemplo: `#define CUADRADO(x) ((x)*(x))` → al escribir `CUADRADO(5)`, el preprocesador lo reemplaza por `((5)*(5))` antes de compilar.

c. Ensamblador

El analizador sintáctico del ensamblador fue implementado utilizando la librería PLY (Python Lex-Yacc), la cual permite definir formalmente la gramática del lenguaje ensamblador mediante reglas de producción. Su función principal consiste en validar que cada instrucción escrita por el usuario cumpla con la sintaxis definida por la arquitectura CacaoCore-64 y, posteriormente, transformar dicha representación textual en código máquina. Para ello, el parser recibe los tokens producidos por el analizador léxico y determina la estructura de cada línea del programa según el tipo de instrucción, operandos y modo de direccionamiento utilizado.

La estructura del parser se encuentra encapsulada dentro de la clase `AsmParser`, la cual mantiene diferentes estructuras internas necesarias para el proceso de ensamblado. Entre ellas se destacan la tabla de símbolos (`symbol_table`), encargada de asociar etiquetas con direcciones dentro de la sección de texto; la tabla de variables (`var_table`), utilizada para almacenar las posiciones de memoria de las variables declaradas; y las listas `program_text`, `program_data` y `program`, que contienen respectivamente las instrucciones de la sección de código, los datos definidos por el usuario y la representación general del programa ensamblado.

La gramática implementada permite reconocer múltiples formatos de instrucciones según los operandos utilizados. De esta manera, el lenguaje ensamblador soporta operaciones entre registros, instrucciones con valores inmediatos, accesos directos a memoria, direccionamiento indirecto mediante registros, referencias a variables y saltos hacia etiquetas. Por ejemplo, la producción:

instruction : MNEMONIC REGISTER COMMA NUMBER

define instrucciones formadas por un mnemónico seguido de un registro y un valor inmediato, como:

`movd r1, 10`

Mientras tanto, reglas como:

instruction : MNEMONIC REGISTER COMMA REGISTER

permiten reconocer instrucciones entre registros, y producciones con corchetes permiten representar accesos a memoria o direccionamiento indirecto, como:

```
load r1, [contador]
```

```
store [r2], r1
```

Internamente, el parser identifica el formato exacto de cada instrucción utilizando sufijos asociados al tipo de operandos. Por ejemplo, `_rr` representa instrucciones registro–registro, `_ri` instrucciones registro–inmediato, `_rm` acceso de registro a memoria y `_mr` acceso de memoria a registro. Estos sufijos se concatenan dinámicamente al nombre del mnemónico para buscar la microinstrucción correspondiente dentro de `MICROINSTRUCTION_SPECS`, permitiendo seleccionar automáticamente el opcode adecuado para cada variante de instrucción.

Una vez identificada la microinstrucción correspondiente, el parser construye la representación binaria utilizando conversiones entre enteros, bytes y cadenas hexadecimales. Durante este proceso se realiza reordenamiento de bytes dependiendo del formato interno definido por la arquitectura, utilizando distintos tamaños de palabra y configuraciones de endianness. Para valores flotantes, el ensamblador utiliza la librería `struct` de Python con el fin de convertir números en punto flotante a su representación binaria de doble precisión.

El parser también implementa un mecanismo de resolución de símbolos para soportar referencias a etiquetas y variables. Cuando una etiqueta es definida, esta se almacena en la tabla de símbolos junto con la dirección actual dentro de la sección `.text`. Si una instrucción referencia una etiqueta que aún no ha sido declarada, la referencia se almacena temporalmente en la lista `pending`, permitiendo resolverla posteriormente una vez que la etiqueta sea encontrada. Este mecanismo resulta esencial para soportar saltos hacia adelante dentro del flujo del programa.

Adicionalmente, durante la construcción de ciertas instrucciones que involucran referencias a memoria o variables, el ensamblador utiliza marcadores temporales dentro de la representación hexadecimal de la instrucción. Un ejemplo de esto es el uso del carácter `-` dentro de cadenas intermedias generadas por el parser. Este símbolo no forma parte de la sintaxis del lenguaje ensamblador visible para el usuario, sino que actúa como un separador interno o placeholder utilizado durante el proceso de ensamblado para distinguir temporalmente campos asociados al opcode, registros y direcciones simbólicas antes de producir el código máquina definitivo.

El lenguaje ensamblador también soporta directivas de sección como `.text` y `.data`, las cuales permiten separar las instrucciones ejecutables de los datos definidos por el usuario. Asimismo, las etiquetas se declaran utilizando un identificador seguido de dos puntos, mientras que las variables se definen mediante un nombre asociado a un valor numérico o flotante. Todas estas construcciones son reconocidas y validadas por el parser mediante reglas gramaticales específicas.

Finalmente, el ensamblador incorpora un sistema de manejo de errores sintácticos mediante la excepción personalizada `ASMParseError`. Cuando ocurre un error durante el análisis, el parser identifica el token problemático, determina la línea exacta donde ocurrió el fallo y muestra el fragmento correspondiente del código fuente. Esto facilita significativamente la depuración de programas ensambladores y mejora la experiencia de desarrollo sobre la arquitectura CacaoCore-64.

Distinción entre etiquetas y variables

El lexer del ensamblador diferencia las etiquetas de salto de las variables de datos por el uso de mayúsculas y minúsculas. Esta regla es fundamental para escribir cualquier programa ensamblador en el proyecto:

- Identificadores en MAYÚSCULAS → son etiquetas de salto (tipo LABEL). Marcan posiciones en el código a las que se puede saltar con instrucciones como `jmp`, `jz` o `call`.
- Identificadores en minúsculas → son variables del segmento de datos (tipo VAR). Se declaran en la sección `.data` y representan posiciones de memoria con nombre.

Si se escribe una etiqueta en minúsculas o una variable en mayúsculas, el lexer las interpretará incorrectamente y el programa no funcionará como se espera.

Comentarios en el ensamblador

En el ensamblador de CacaoCore-64, los comentarios se marcan con punto y coma (;) o con el símbolo numeral (#). Todo lo que viene después de cualquiera de esos dos caracteres hasta el fin de la línea se ignora. Esto es distinto al lenguaje de alto nivel CacaoScript, donde los comentarios se escriben con `//`.

d. Enlazador-Cargador

El enlazador-cargador representa la etapa final del proceso de traducción. Su función principal es integrar múltiples módulos objeto en un único binario ejecutable y posteriormente cargar dicho binario en memoria para su ejecución.

Desde la perspectiva de análisis, se identificaron dos responsabilidades claramente diferenciadas: el enlazado y el cargado. El enlazado implica la resolución de referencias externas entre módulos, mientras que el cargado consiste en la ubicación del código y los datos en memoria, así como la inicialización del entorno de ejecución.

El diseño del enlazador incluye la construcción de una tabla global de símbolos, obtenida a partir de la combinación de las tablas locales de cada módulo. Esta tabla permite detectar errores como símbolos duplicados o no definidos, y facilita la asignación de direcciones finales a cada referencia. Asimismo, se implementa un proceso de reubicación, mediante el cual se ajustan las direcciones de memoria en función de la posición final del código.

En cuanto al cargador, su diseño contempla la validación de los límites de memoria, la copia de las secciones de código y datos en las direcciones correspondientes, y la inicialización de registros clave del procesador, como el contador de programa y el puntero de pila.

La implementación de este componente garantiza que los programas enlazados puedan ejecutarse correctamente en la arquitectura CacaoCore-64, cerrando así el ciclo completo de traducción.

e. Módulo de análisis léxico del compilador

El módulo de análisis léxico constituye la primera fase del compilador y tiene como objetivo transformar el código fuente en una secuencia de tokens. Este módulo ha sido diseñado e implementado utilizando especificaciones basadas en el metalenguaje FLEX, lo que permite definir de manera formal los patrones de reconocimiento del lenguaje.

i. Vocabulario del compilador

Como parte del análisis, se determinó el vocabulario completo que reconoce el compilador. Este incluye palabras clave del lenguaje (como estructuras de control, definiciones y operaciones), tipos de datos primitivos, identificadores, operadores aritméticos y relacionales, delimitadores y literales.

Las palabras clave abarcan construcciones como declaración de variables, funciones, condicionales y ciclos. Los tipos de datos incluyen enteros, flotantes, cadenas y booleanos. Los identificadores siguen reglas formales de formación, mientras que los operadores y delimitadores permiten estructurar expresiones y sentencias.

Asimismo, se reconocen literales numéricos, cadenas de texto y valores booleanos, así como comentarios que son ignorados durante el análisis.

ii. Categorías gramaticales

A partir del vocabulario definido, se establecieron las categorías gramaticales necesarias para la construcción del compilador.

Las categorías léxicas corresponden a los tokens generados por el analizador léxico, tales como identificadores, literales, operadores y palabras clave.

Esta separación permite construir una gramática formal del lenguaje SPL, la cual es utilizada posteriormente en el análisis sintáctico para generar el árbol de sintaxis abstracta.

iii. Especificación de patrones léxicos

En la etapa de diseño, se definieron los patrones de reconocimiento de tokens mediante expresiones regulares. Estas expresiones permiten identificar de manera precisa cada categoría léxica.

Por ejemplo, los identificadores se definen mediante una expresión que exige una letra o guion bajo inicial, seguida de caracteres alfanuméricos. Los números enteros y flotantes se reconocen mediante patrones numéricos específicos, mientras que las cadenas de texto se delimitan por comillas. Los operadores y delimitadores se identifican mediante expresiones simples, teniendo en cuenta la precedencia en el caso de operadores compuestos.

En la implementación, estos patrones se traducen en reglas léxicas dentro del analizador, permitiendo que el sistema procese el código fuente de manera eficiente y estructurada. Adicionalmente, se incorporan mecanismos para ignorar espacios en blanco y comentarios, así como para llevar un control del número de línea.

Además de los operadores aritméticos, de comparación y lógicos listados, el analizador léxico reconoce dos operadores que no aparecen en la descripción anterior:

- `xor` — operador lógico O exclusivo. Devuelve verdadero cuando exactamente uno de los dos operandos es verdadero. Ejemplo: `let c : bool = a xor b`.
- `+=` (token `SWEET_PLUS`) — operador de asignación compuesta. Suma un valor a una variable y guarda el resultado sin tener que escribir la variable dos veces. Ejemplo: `set contador += 1` equivale a `set contador = contador + 1`.

Tabla de números

Además de la tabla de símbolos (para identificadores), el analizador léxico mantiene una segunda tabla llamada tabla de números. Por cada literal numérico encontrado en el

código, esta tabla registra: el valor ya convertido a número (no como texto), si es entero o decimal, y en qué líneas aparece. Esta tabla es visible en la interfaz gráfica junto con la tabla de símbolos al ejecutar el análisis léxico.

f. Análisis Sintáctico

El parser utiliza PLY-Yacc y una gramática libre de contexto para construir el AST directamente durante el reconocimiento sintáctico.

El analizador sintáctico recibe como entrada la secuencia de tokens que produjo el analizador léxico y verifica que esa secuencia forme un programa válido según las reglas de CacaoScript. Cuando la verificación es exitosa, el resultado es un Árbol de Sintaxis Abstracta (AST): una estructura en memoria donde cada nodo representa una parte del programa (una declaración, una operación, un bloque de código, etc.). Si se detectan errores, se registran con el número de línea y el token problemático, y el parser intenta continuar para reportar tantos problemas como sea posible en una sola pasada.

La implementación usa PLY (Python Lex-Yacc) con el algoritmo LALR(1). Esto significa que el parser lee el código de izquierda a derecha, toma decisiones con solo un token de anticipación y resuelve la estructura del programa sin necesidad de retroceder. La gramática está definida como métodos Python cuyo docstring contiene las producciones en formato BNF, patrón propio de PLY. Cada regla construye el nodo de AST correspondiente y lo devuelve hacia arriba en la jerarquía hasta que se ensambla el árbol completo.

La gramática cubre todas las construcciones del lenguaje CacaoScript:

- Declaraciones de variables con `let`, incluyendo arreglos de una o dos dimensiones y valor inicial.
- Reasignaciones con `set`, usando asignación simple (`=`) o acumulativa (`+=`).
- Definición de funciones con `func`, con parámetros tipados y bloque de código.
- Definición de estructuras personalizadas con `mold` (similar a clases).
- Control de flujo: condicional `if / otherwise`, ciclo `asLongAs`, ciclo `for`.
- Instrucciones `show` (imprimir), `deliver` (retornar valor) y `oops` (lanzar error).
- Expresiones aritméticas, de comparación y lógicas, con precedencia y asociatividad correctas.

- Acceso a elementos de arreglo (`arreglo[i]`) y a atributos de estructuras (`objeto.campo`).
- Llamadas a funciones y métodos, creación de instancias con `summon`, referencia al objeto actual con `ohmy`.

Para evitar ambigüedades al mezclar operadores, el parser define una tabla de precedencia explícita (de menor a mayor): primero `or`, luego `and`, `not`, operadores de igualdad (`==` `!=`), operadores de comparación (`<` `>` `<=` `>=`), suma y resta, multiplicación/división/módulo, negación unaria (`-x`), acceso a arreglo (`[]`) y finalmente acceso a miembro (`.`). Esta tabla le dice al parser cómo agrupar las expresiones sin que el programador tenga que escribir paréntesis extra.

Cuando el parser encuentra un token que no encaja en ninguna regla válida, registra el error con la línea y el nombre del token, luego descarta ese token y reinicia el análisis desde el siguiente punto estable (mecanismo de recuperación de PLY). Esto permite que una sola ejecución detecte múltiples errores en lugar de detenerse al primero.

Las reglas implementadas soportan:

Declaraciones.

Funciones.

Expresiones.

Ciclos.

Condicionales.

Moldes.

Acceso a atributos.

Retornos.

La implementación incluye precedencia y asociatividad de operadores para evitar conflictos `shift/reduce`.

El parser produce nodos especializados que posteriormente son recorridos por el analizador semántico.

i. Jerarquía de nodos del Árbol Sintáctico Abstracto (AST)

Cuando el analizador sintáctico lee el código fuente de CacaoScript, no produce texto plano: construye un árbol en memoria que representa la

estructura lógica del programa. Cada nodo de ese árbol corresponde a una parte del código, por ejemplo una declaración, una operación o una función. Ese árbol se llama Árbol Sintáctico Abstracto (AST).

El archivo `ast_nodos.py` define todas las clases de nodos que puede tener ese árbol. Esto es fundamental para las fases siguientes (semántico y generador de código), porque trabajan recorriendo ese árbol nodo por nodo.

Nodos principales

Nodos estructurales

Representan la forma general del programa.

- `NodoPrograma`: representa el programa completo; contiene la lista de todas las instrucciones.
- `NodoBloque`: representa un bloque de código entre llaves `{ }`.

Nodos de declaración y asignación

- `NodoDeclaracion`: corresponde a `let nombre : tipo = valor`.
- `NodoReasignacion`: corresponde a `set nombre = valor` o `set nombre += valor`.

Nodos de control de flujo

- `NodoSi`: representa `if ... { } otherwise { }`. Guarda la condición, el bloque verdadero y el bloque alternativo (opcional).
- `NodoMientras`: representa `asLongAs condición { }`.
- `NodoPara`: representa `for (inicio, condición, actualización) { }`.

Nodos de funciones y estructuras

- `NodoFuncion`: define una función con su nombre, lista de parámetros y su bloque de código.
- `NodoMold`: define una estructura tipo clase con sus atributos y métodos.
- `NodoSummon`: crea una nueva instancia de un mold (equivalente a `new` en otros lenguajes).

Nodos de expresión

- **NodoBinario:** cualquier operación entre dos valores: suma, resta, comparación, etc. Guarda el operador y los dos operandos.
- **NodoUnario:** operaciones sobre un solo valor: negación numérica (-x) o negación lógica (not x).
- **NodoLlamada:** llamada a una función con su nombre y la lista de argumentos.
- **NodoAccesoMiembro:** acceso a un atributo o método de una estructura, por ejemplo objeto.campo.
- **NodoAccesoArreglo:** acceso a un elemento de un arreglo, por ejemplo arreglo[índice].

Nodos literales

Representan valores constantes escritos directamente en el código.

- **NodoEntero:** un número entero, por ejemplo 42.
- **NodoFlotante:** un número decimal, por ejemplo 3.14.
- **NodoCadena:** una cadena de texto entre comillas.
- **NodoBooleano:** un valor verdadero (indeed) o falso (nope).
- **NodoNada:** el valor vacío nothing (equivalente a null).

Nodos de acción

- **NodoEntregar:** instrucción deliver; retorna un valor desde una función.
- **NodoMostrar:** instrucción show; imprime un valor en la consola.
- **NodoOops:** instrucción oops; lanza un error o excepción.

Esta jerarquía permite que el semántico y el generador de código traten cada construcción del lenguaje de forma independiente y ordenada, recorriendo el árbol de arriba a abajo.

g. Análisis Semántico

Una vez que el analizador sintáctico construye el AST y confirma que el programa tiene una estructura válida, el análisis semántico se encarga de verificar

que ese programa también tenga sentido. La estructura puede ser correcta y aun así el código puede ser erróneo: usar una variable que nunca se declaró, sumar un número con un texto, llamar a una función con más argumentos de los que acepta. Esos errores no los detecta la gramática; los detecta el análisis semántico.

El análisis semántico de CacaoCore-64 está organizado en dos módulos que trabajan en secuencia. El primero revisa el código y detiene el proceso si algo no tiene sentido. El segundo, que solo actúa si el primero no encontró errores, enriquece el AST con información adicional que el generador de código necesita para producir instrucciones ensamblador correctas.

Analizador Semántico (`analizador_semantico.py`)

Este módulo toma el código fuente (no el AST, sino el texto original) y lo analiza con su propia gramática usando PLY yacc. Su única responsabilidad es encontrar errores de significado y reportarlos. Si detecta alguno, el proceso se detiene y el error se muestra al usuario con el número de línea y el identificador involucrado. Entre los errores que detecta se encuentran: variables usadas antes de declararse, tipos incompatibles en una operación o asignación, funciones llamadas con un número incorrecto de argumentos, redeclaración de un identificador ya existente en el mismo ámbito, y acceso a atributos de un `mod` que no existen.

Anotador Semántico (`semantic_ast_annotator.py`)

Este módulo recorre el AST que produjo el parser sintáctico y agrega información extra a cada nodo: el tipo de dato de cada expresión, el ámbito (`scope`) al que pertenece cada variable y los parámetros que espera cada función. Esta información queda guardada dentro de los propios nodos del árbol, de modo que cuando el generador de código visite cada nodo ya sabe de antemano qué tipo de valor maneja y en qué contexto.

Durante el recorrido se realizan validaciones contextuales utilizando:

Tabla de símbolos.

Contextos de alcance.

Tipos inferidos.

Firmas de funciones.

Entre las verificaciones implementadas se encuentran:

Uso previo de variables.

Compatibilidad de tipos.

Redeclaraciones.

Validación de retornos.

Validación de parámetros.

Resolución de scopes.

Existencia de atributos.

Compatibilidad de operaciones.

Los errores semánticos son reportados indicando:

Tipo de error.

Línea.

Identificador asociado.

Descripción contextual.

i. Soporte para estructuras tipo clase: mold, summon y ohmy

CacaoScript permite definir estructuras personalizadas que agrupan datos y funciones relacionadas, de forma similar a las clases en otros lenguajes.

Para esto se usan tres palabras clave nuevas: mold, summon y ohmy.

mold: definir una estructura

Con mold se define un nuevo tipo de dato que puede tener atributos (campos) y métodos (funciones propias). Es como crear un molde o plantilla que luego se puede usar para crear objetos concretos.

None

```
mold Punto {  
  let x : int  
  let y : int  
  
  func mover(dx : int, dy : int) {  
    set ohmy.x = ohmy.x + dx  
    set ohmy.y = ohmy.y + dy  
  }  
}
```

```
}
```

summon: crear una instancia

summon se usa para crear un objeto a partir de un mold, reservando espacio en memoria para sus atributos.

None

```
let p : Punto = summon Punto()
```

ohmy: referencia al objeto actual

Dentro de un método de un mold, ohmy es la referencia al objeto sobre el que se está ejecutando ese método. Es equivalente a this o self en otros lenguajes.

None

```
set ohmy.x = 5    // modifica el atributo x del objeto actual
```

Que valida el analizador semántico para los molds

- Que el mold exista antes de usarse.
- Que el atributo al que se accede con el punto (.) realmente pertenezca al mold.
- Que el tipo del valor asignado a un atributo sea compatible con el tipo declarado.
- Que los métodos invocados sobre un objeto existan en su mold.

Si cualquiera de estas validaciones falla, el compilador reporta un error semántico indicando exactamente qué salió mal y en qué línea.

ii. Anotador semántico del AST

El proceso de análisis semántico en CacaoCore-64 está dividido en dos partes que trabajan juntas:

- El Analizador Semántico: recorre el código fuente y detecta errores de significado, como tipos incompatibles o variables no declaradas.
- El Anotador Semántico (`semantic_ast_annotator.py`): toma el AST ya construido por el parser y lo enriquece con información extra que el generador de código va a necesitar.

¿ Por qué es necesario el anotador?

El árbol sintáctico (AST) que produce el parser contiene la estructura del programa, pero no sabe de qué tipo es cada expresión. El anotador recorre ese árbol y le agrega esa información directamente en cada nodo.

Por ejemplo, cuando encuentra la expresión `3 + 4.0`, el anotador anota ese nodo como resultado de tipo `float`, porque la suma de un entero y un flotante produce un flotante. El generador de código luego usa esa anotación para elegir la instrucción de punto flotante correcta.

¿ Qué información agrega el anotador?

- El tipo de cada expresión: `int`, `float`, `text`, `bool`, o el nombre de un `mold`.
- A qué ámbito (`scope`) pertenece cada variable: si es local a una función o global.
- Si un nodo representa un atributo de un `mold`, guarda a que `mold` pertenece.
- Si una llamada de función es válida, guarda la firma (nombre y tipos de parámetros) del destino.

Flujo general

El orden en que se usan estos componentes es:

None

Codigo fuente

-> Parser (construye AST)

```
-> Analizador Semantico (detecta errores)
-> Anotador Semantico (enriquece el AST con tipos y scopes)
-> Generador de Codigo (usa el AST anotado para producir
ensamblador)
```

h. Generador de Código

La generación de código se realiza recorriendo el AST y produciendo instrucciones al ensamblador.

El generador mantiene:

Contadores de etiquetas.

Gestión de temporales.

Contextos de función.

Manejo de stack.

Las estructuras de control son traducidas mediante etiquetas y saltos condicionales.

Las llamadas a función generan:

Paso de parámetros.

Actualización de stack.

Manejo de retorno.

Restauración de contexto.

i. Gestor de registros.

Cuando el generador de código transforma el AST en instrucciones de ensamblador, necesita guardar valores temporales en algún lugar. En la arquitectura CacaoCore-64 esos lugares son los registros del procesador (R0 a R12).

El Gestor de Registros (`gestor_de_registros.py`) se encarga de administrar los registros que están libres y cuales están en uso en cada momento de la generación de código.

¿ Cómo funciona?

El gestor mantiene una lista de registros disponibles (R0 hasta R12). Cuando el generador necesita guardar un resultado temporal:

- Pide un registro libre al gestor (operación "adquirir").
- El gestor entrega el siguiente disponible y lo marca como ocupado.
- Cuando el generador ya no necesita ese valor, devuelve el registro al gestor (operación "liberar").
- El gestor lo marca nuevamente como libre para que otro cálculo pueda usarlo.

Registros reservados (el gestor nunca los entrega)

Algunos registros tienen uso fijo y no se usan como temporales:

Registro	Uso fijo
R13 (SP)	Stack Pointer: apunta al tope de la pila.
R14 (LR)	Link Register: guarda la direccion de retorno en llamadas a funcion.
R15 (ACC)	Acumulador: la ALU deposita aqui los resultados de las operaciones.

i. Conclusiones

El desarrollo presentado en este documento demuestra la implementación de los componentes fundamentales de un sistema de compilación completo. En particular, se ha logrado el diseño e implementación del preprocesador, el ensamblador, el enlazador-cargador y el módulo de análisis léxico del compilador, todos ellos basados en principios formales y herramientas equivalentes al metalenguaje FLEX.

Cada uno de estos componentes cumple un rol específico dentro del sistema, y su integración permite transformar programas escritos en SPL en código ejecutable para la arquitectura CacaoCore-64. El diseño modular adoptado facilita la extensión futura del sistema, especialmente en las fases de análisis sintáctico, análisis semántico y generación de código.

Con el desarrollo del Taller 2, el sistema CacaoCore-64 cuenta con un compilador completo de cuatro fases: análisis léxico, análisis sintáctico, análisis semántico y generación de código. El Árbol Sintáctico Abstracto actúa como pieza central que conecta todas las fases entre sí. El sistema es capaz de compilar programas escritos en CacaoScript de punta a punta, desde el código fuente hasta instrucciones ejecutables en el procesador simulado. Se implementó además soporte para programación orientada a objetos básica mediante las palabras clave `mold`, `summon` y `ohmy`. Como trabajo futuro quedan: optimización del código generado, soporte para más tipos de datos y mejoras en los mensajes de error.

8. Experimentación y Análisis de Resultados

Para probar la máquina se ejecutaron dos archivos uno para cada parte, el primero dedicado únicamente al flujo de proceso a lexic análisis y el otro al de assembler hasta loader and linker, los archivos usados se muestran a continuación:

Flujo preprocessor - Lexical Analysis

```
// Programa de prueba
```

```
#define MIN_AGE 18
#define TRESHOLD 30
```

```
func greet(name: text) {
  show "Hello, " + name
  deliver nothing
}
```

```
let age: int = 25
let seconds: height = 180.0
let active: bool = indeed
```

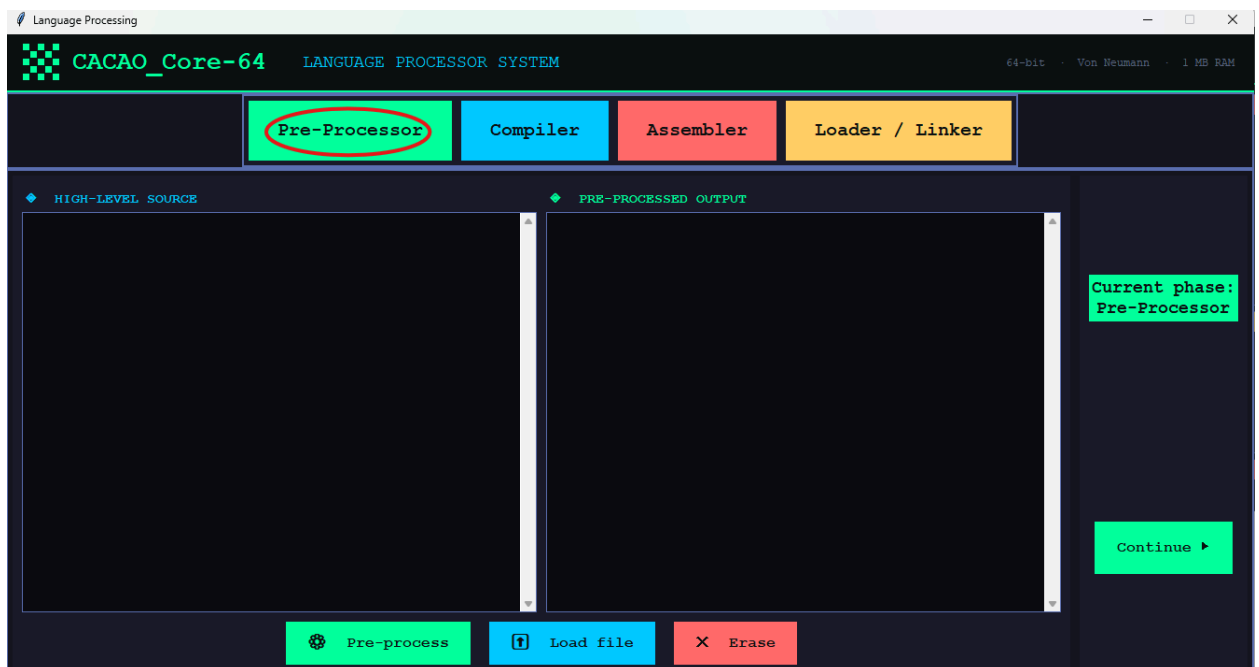
```
if age > MIN_AGE {
  show "Welcome."
} otherwise {
  set active = nope
  show "Not today."
}
```

```
asLongAs age < TRESHOLD {
```

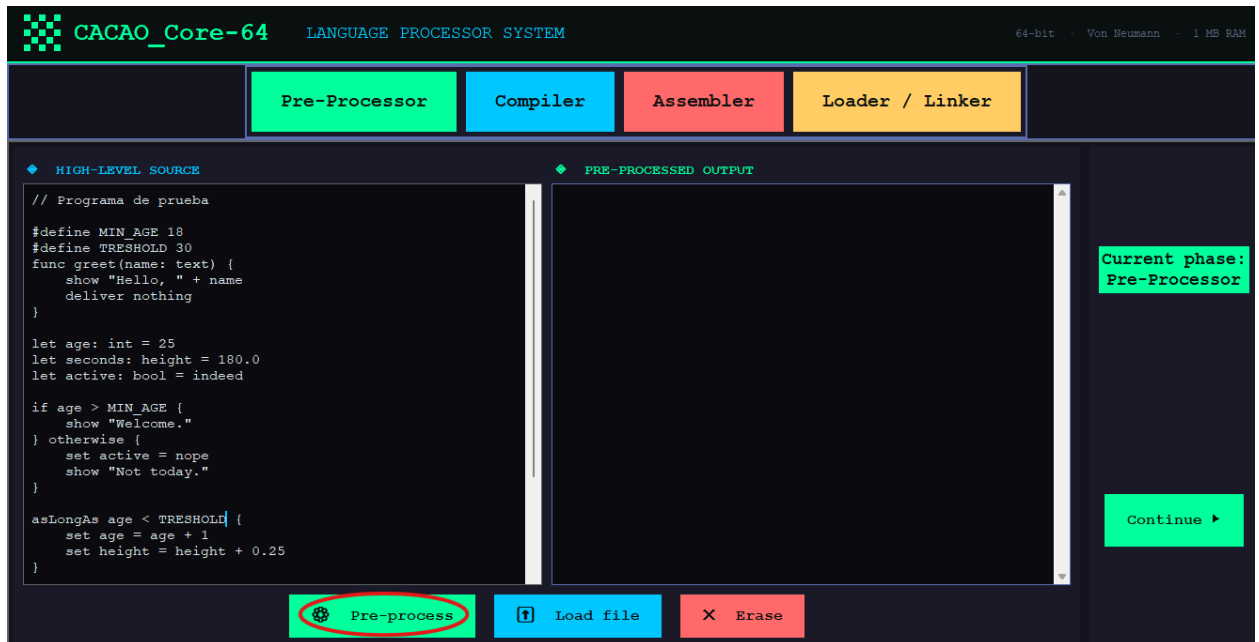
```
    set age = age + 1
    set height = height + 0.25
}
```

```
show age
show height
```

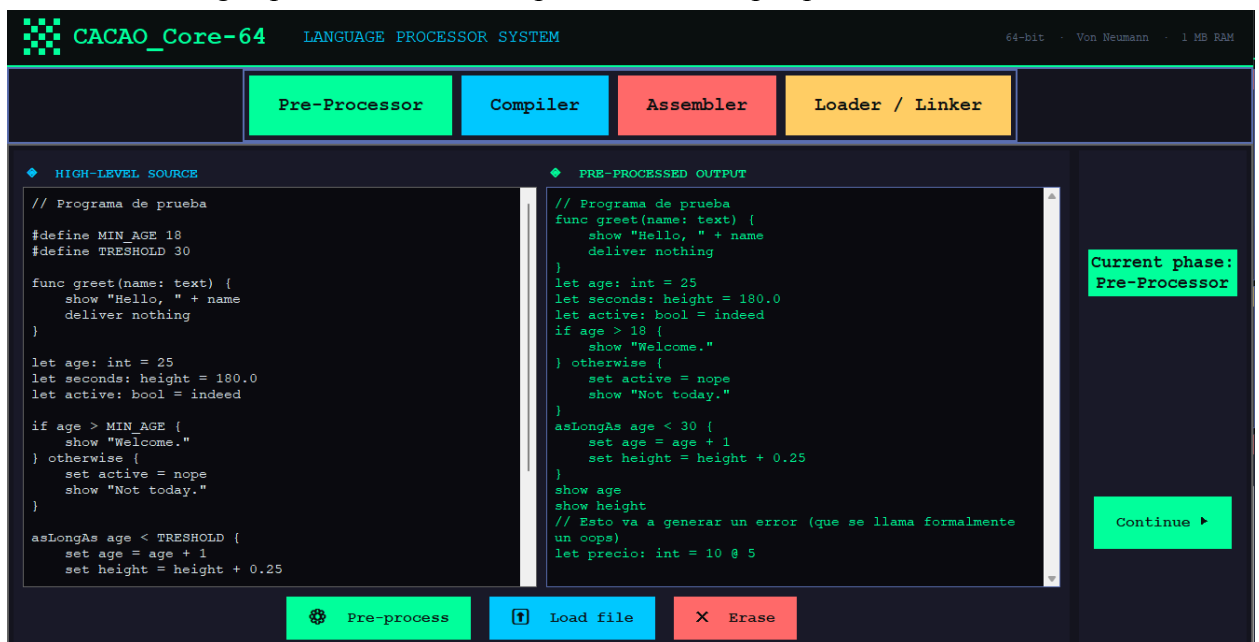
```
// Esto va a generar un error (que se llama formalmente un oops)
let precio: int = 10 @ 5
```



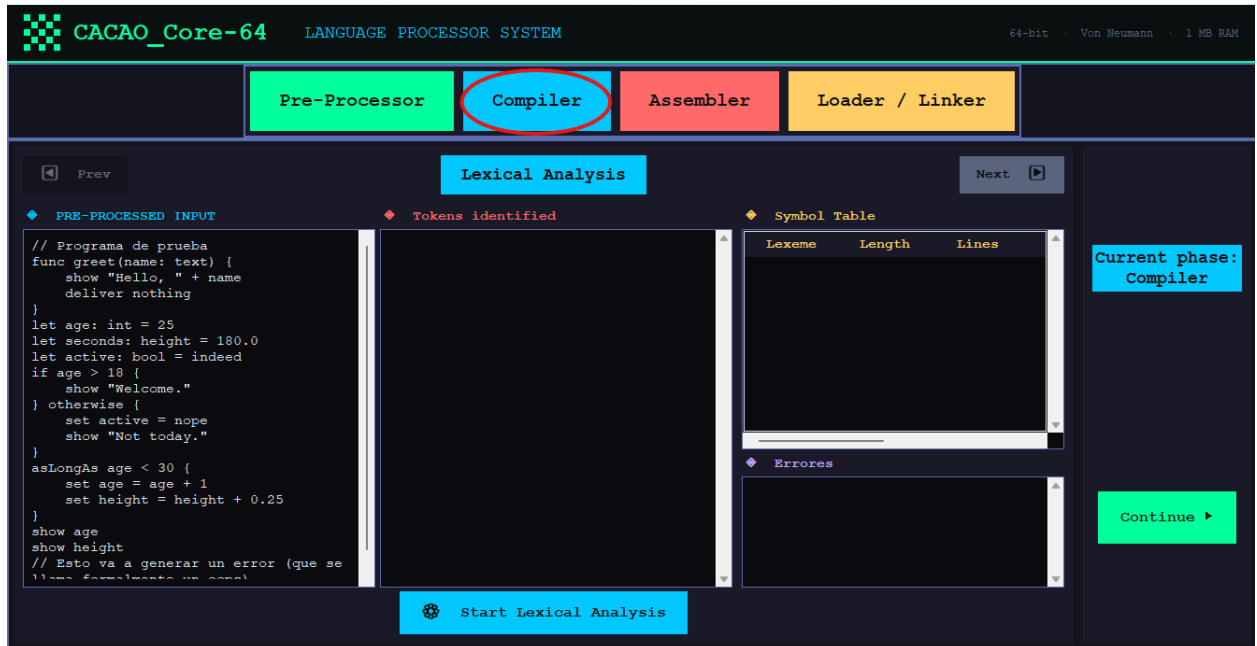
Para comenzar se sube el archivo presionando el botón load file y escogiendo el archivo que sea de interés o si se desea escribir directamente el código en el campo:



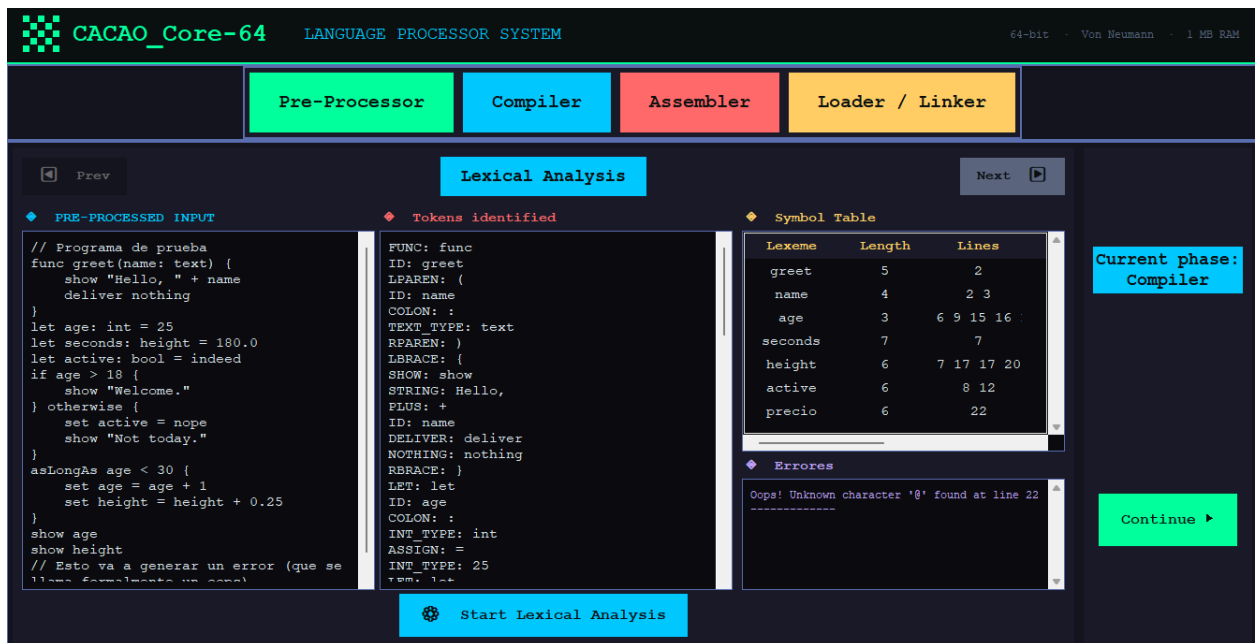
Ya con el código a procesar en el campo izquierdo se presiona el botón encerrado en la imagen para así ver la correspondiente salida pre-procesada:



Esta salida está lista para pasar por el proceso de compilación, para acceder a la siguiente fase del procesamiento presione el botón “continue” ubicado a la derecha de la interfaz. Como se puede observar los **defines** fueron reemplazados correctamente en el código de salida



En esta segunda fase el campo de más a la izquierda contiene el código fuente a compilar producto de la fase de preprocesamiento, para empezar el análisis léxico se presiona con el campo input ya cargado el botón “Start Lexical Analysis”:



El resultado de este análisis léxico es todos los tokens encontrados en el campo tokens identified, la tabla de símbolos actualizada con los lexemas, su longitud y las líneas donde aparecen y una ventana de error que informa de tokens que no pudieron ser reconocidos.

Flujo completo - Inclusión de Análisis Sintáctico, Semántico y Generación

```
let n: int = 5
```

```
let a: int[5] = 1,8,4,2,10
```

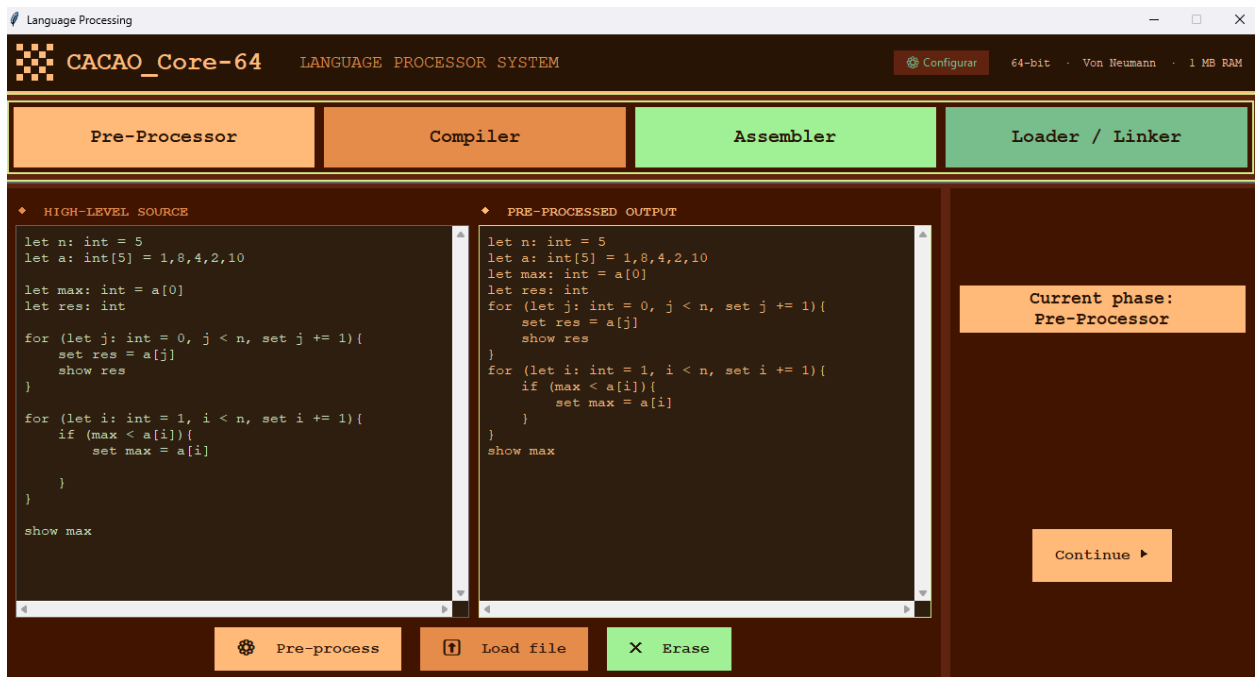
```
let max: int = a[0]
```

```
let res: int
```

```
for (let j: int = 0, j < n, set j += 1){  
    set res = a[j]  
    show res  
}
```

```
for (let i: int = 1, i < n, set i += 1){  
    if (max < a[i]){  
        set max = a[i]  
    }  
}
```

```
show max
```



Language Processing

CACAO_Core-64 LANGUAGE PROCESSOR SYSTEM

Configurar 64-bit · Von Neumann · 1 MB RAM

Pre-Processor Compiler Assembler Loader / Linker

Prev Lexical Analysis Next

PRE-PROCESSED INPUT

```
let n: int = 5
let a: int[5] = 1,8,4,2,10
let max: int = a[0]
let res: int
for (let j: int = 0, j < n, set j
    set res = a[j]
    show res
)
for (let i: int = 1, i < n, set i
    if (max < a[i]){
        set max = a[i]
    }
)
show max
```

Tokens identified

```
LET: let
ID: n
COLON: :
INT_TYPE: int
ASSIGN: =
INT_LIT: 5
LET: let
ID: a
COLON: :
INT_TYPE: int
LBRACKET: [
INT_LIT: 5
RBRACKET: ]
ASSIGN: =
INT_LIT: 1
COMMA: ,
INT_LIT: 8
COMMA: ,
INT_LIT: 4
COMMA: ,
```

Symbol Table

Lexeme	Length	Lines
n	1	1 5 9
a	1	2 3 6 10
max	3	3 10 11

Numeric Symbol Table

Lexeme	Type	Value
5	int	5
1	int	1
8	int	8

Errores

Current phase: Compiler

Continue

Start Lexical Analysis

Language Processing

CACAO_Core-64 LANGUAGE PROCESSOR SYSTEM

Configurar 64-bit · Von Neumann · 1 MB RAM

Pre-Processor Compiler Assembler Loader / Linker

Prev Syntactic Analysis Next

AST Created

```
program
├── sentencias: list[7]
│   ├── [0]: let_stmt
│   │   ├── let_keyword: "let"
│   │   ├── nombre: "n"
│   │   ├── colon: ":"
│   │   ├── tipo: "int"
│   │   ├── assign: "="
│   │   └── valor: INT_LIT
│   │       └── valor: 5
│   ├── [1]: let_stmt
│   │   ├── let_keyword: "let"
│   │   ├── nombre: "a"
│   │   ├── colon: ":"
│   │   ├── tipo: "int"
│   │   ├── lbracket1: "["
│   │   ├── dim1: INT_LIT
│   │   │   └── valor: 5
│   │   ├── rbracket1: "]"
│   │   └── assign: "="
│   └── [2]: let_stmt
│       ├── let_keyword: "let"
│       ├── nombre: "res"
│       ├── colon: ":"
│       ├── tipo: "int"
│       ├── assign: "="
│       └── valor: INT_LIT
│           └── valor: 0
```

Symbol Table

Lexeme	Length	Lines	Kind
n	1	1 5 9	None
a	1	2 3 6 10 1	None
max	3	3 10 11 14	None
res	3	4 6 7	None

Numeric Symbol Table

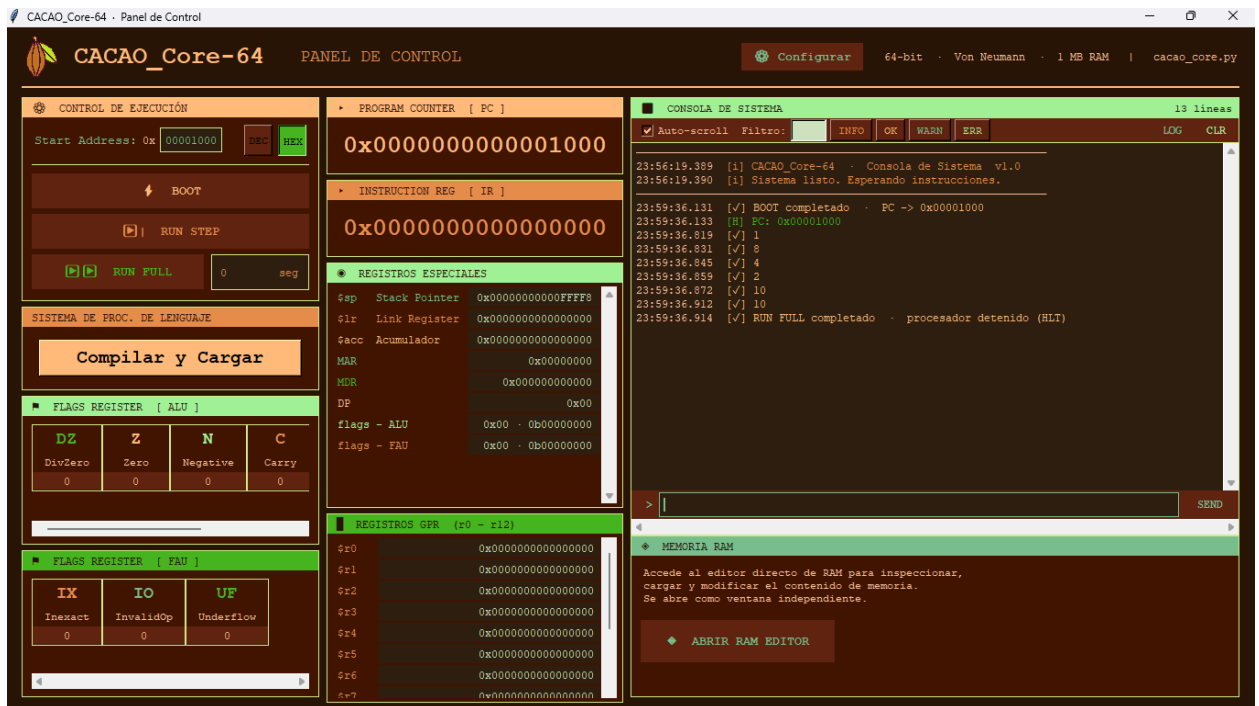
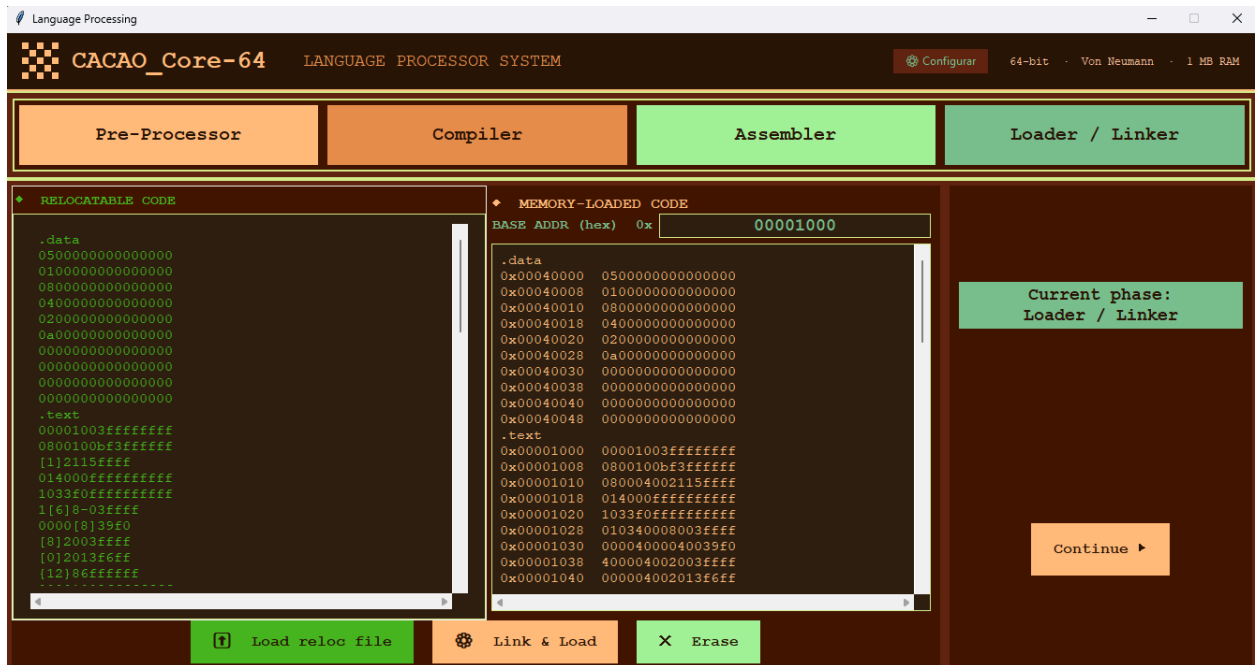
Lexeme	Type	Value	Line
5	int	5	2
1	int	1	9
8	int	8	2
4	int	4	2

Errores

Current phase: Compiler

Continue

Start Syntactic Analysis



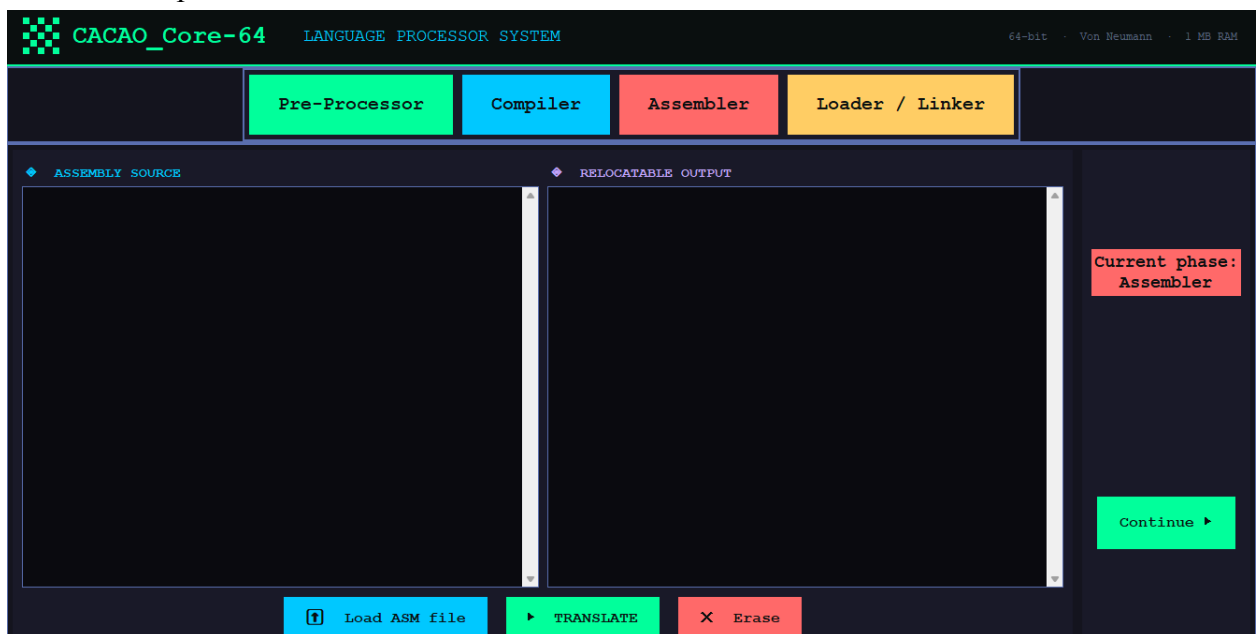
La ejecución se completó satisfactoriamente, con los resultados de la lista y el máximo.

Flujo assembler - Loader / Linker

.data

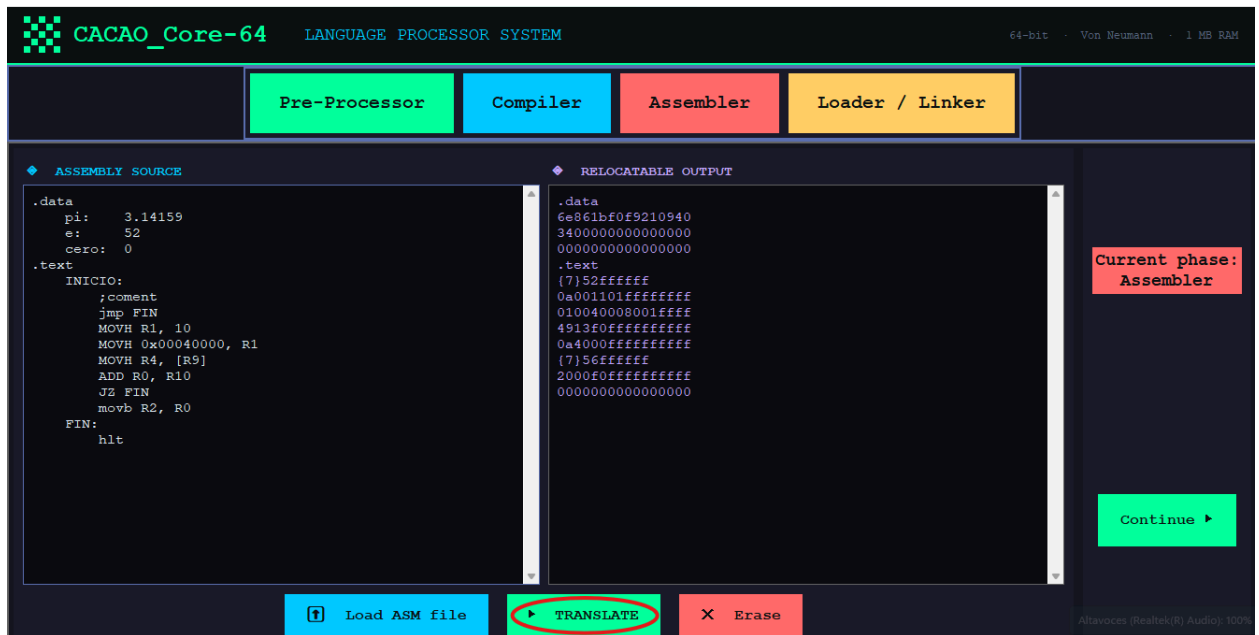

```
pi: 3.14159
e: 52
cero: 0
.text
INICIO:
    ;coment
    jmp FIN
    MOVH R1, 10
    MOVH R1, [cero]
    MOVH [cero], R1
    MOVH 0x00040000, R1
    MOVH R4, [R9]
    ADD R0, R10
    JZ FIN
    movb R2, R0
FIN:
    hlt
```

Esta fase para esta versión del proyecto ha de ser accedida mediante su botón correspondiente “Assembler”:

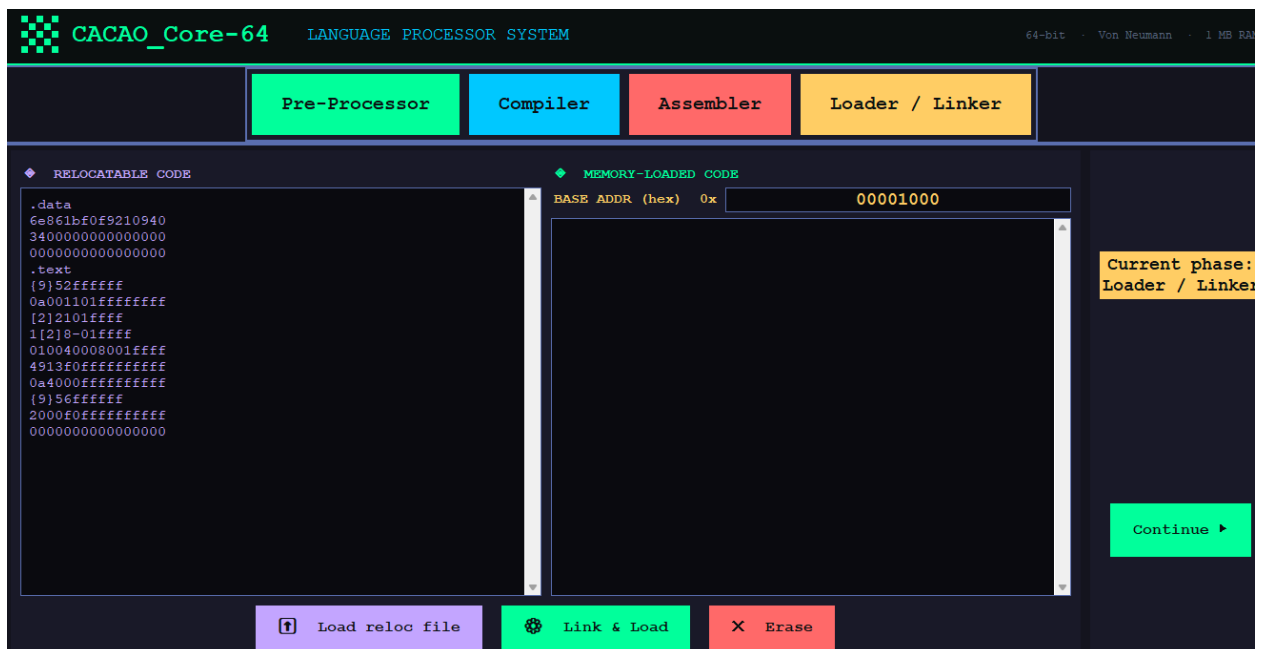


En la interfaz habrá dos campos uno con el código en asm y otro que recibirá el output de código relocable correspondiente, para subir un archivo a él source

haga uso del botón “Load ASM file” o escriba el código asm deseado directamente:

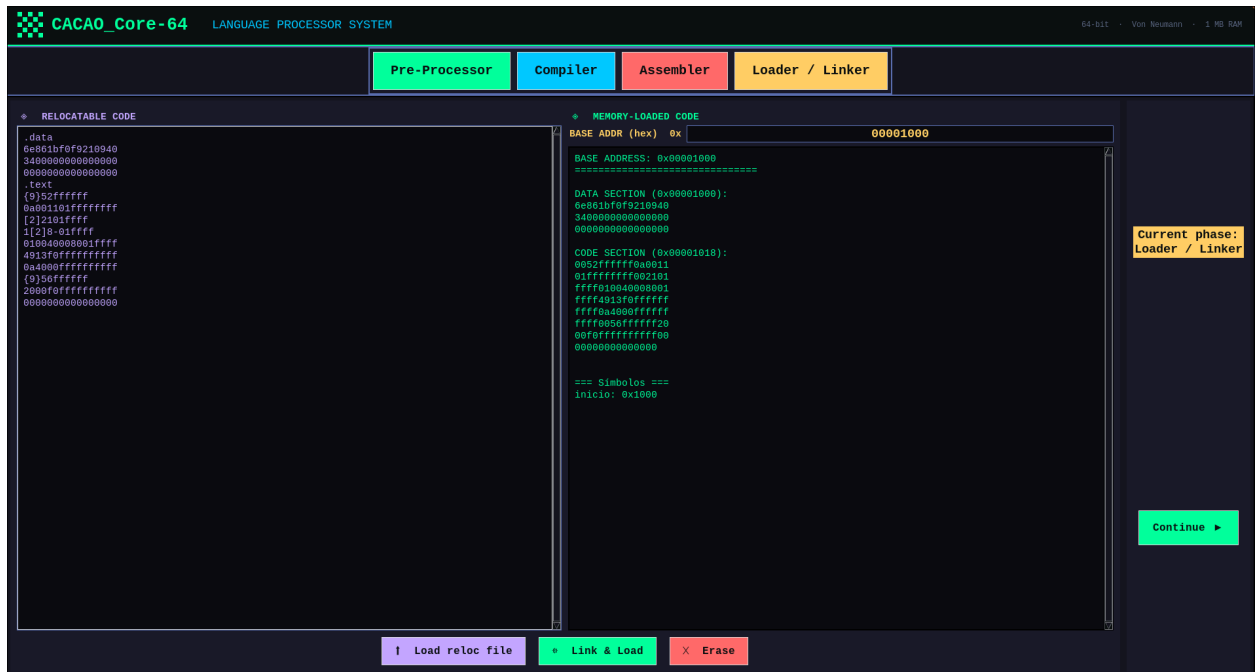


Con el código a traducir en el campo izq presione el botón encerrado “TRANSLATE” con el cual producirá el código reloj resultante como se muestra en la imagen previa, para continuar con el flujo del procesamiento presione continue una vez finalizada la etapa.



En esta última etapa dada una dirección de memoria escogida previamente en el entry de la sección se calcularán las direcciones absolutas para todas aquellas

referencias relativas y posteriormente se escribirá todo el código binario resultante en la ram desde la debida posición de memoria.



Casos de prueba del analizador semántico

El proyecto incluye una carpeta dedicada a probar que el analizador semántico detecta correctamente los errores. Hay dos tipos de archivos de prueba:

- Un ejemplo correcto (correct_example.txt): un programa sin errores que debe compilar sin problemas.
- Trece ejemplos de error: cada uno diseñado para provocar un error semántico específico.

Ejemplo correcto

El archivo correct_example.txt contiene un programa completo que usa variables, funciones, arreglos y estructuras mold, todo de forma válida. Sirve para verificar que el semántico no genera falsos positivos.

Casos de error

Cada archivo de error prueba una situación específica:

Archivo	Que prueba	Ejemplo del error
arr_index.txt	Índice fuera del rango del arreglo	arreglo[20] cuando el arreglo tiene solo 10 elementos
bad_args.txt	Número incorrecto de argumentos en una llamada	func suma(a,b) llamada con 3 argumentos
bad_dims.txt	Dimensiones de arreglo incorrectas	let m : int[3][3] usado como si fuera de 1D
bad_ops.txt	Operación entre tipos incompatibles	sumar un int con un text
bad_set.txt	Asignación de tipo incompatible	asignar un float a una variable int
deliver_types.txt	Tipo de retorno incorrecto	función que declara retornar int pero retorna text
member_access.txt	Acceso a atributo inexistente en un mold	objeto.campoQueNoExiste
method_access.txt	Llamada a método inexistente en un mold	objeto.metodoQueNoExiste()
missing_delivers.txt	Función sin instrucción deliver	func con tipo de retorno pero sin deliver al final
ohmy_scope.txt	Uso de ohmy fuera de un mold	ohmy.campo usado en una función global
over_index.txt	Indexado en variable que no es arreglo	variable_simple[0]

summon_args.txt	Argumentos incorrectos al instanciar un mold	summon Punto(1,2,3) cuando Punto no acepta argumentos
unreachable.txt	Código inalcanzable después de deliver	instrucciones escritas después de un deliver
vars_other.txt	Variable usada antes de ser declarada	usar x antes de let x : int

9. Bibliografía

- Compilers: Principles, Techniques, and Tools. A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson, 2006.
- PLY (Python Lex-Yacc). D. Beazley, *PLY (Python Lex-Yacc)*. Disponible en: <http://www.dabeaz.com/ply>

10. Apéndice

A continuación, se encuentra la gramática del lenguaje y los diagramas de sintaxis correspondientes.

`program ::= statement*`

`statement ::= let_stmt`

`| set_stmt`

`| func_def`

`| mold_def`

`| if_stmt`

`| while_stmt`

`| for_stmt`

`| deliver_stmt`

`| show_stmt`

`| oops_stmt`

`| expr_stmt`

`let_stmt ::= "let" ID ":" type_annot opt_dims opt_initializer`

`opt_dims ::= dims`

`|`

`dims ::= "[" expr "]" ("[" expr "]")*`

`opt_initializer ::= "=" initializer`

`|`

initializer ::= expr
 | value_list

value_list ::= expr "," expr ("," expr)*

set_stmt ::= "set" lvalue ("=" | "+=") expr

lvalue ::= primary_lvalue ("[" expr "]" | "." ID)*

primary_lvalue ::= ID
 | "ohmy"

type_annot ::= "int"
 | "float"
 | "text"
 | "bool"
 | ID

func_def ::= "func" ID "(" param_list? ")" block

param_list ::= param ("," param)*

$\text{param} ::= \text{ID} ":" \text{type_annot} \text{opt_dims}$

$\text{mold_def} ::= \text{"mold"} \text{ID} "{" \text{mold_member}^* \text{"}"}$

$\text{mold_member} ::= \text{let_stmt}$

$\quad | \text{func_def}$

$\text{if_stmt} ::= \text{"if"} \text{expr} \text{block} (\text{"otherwise"} \text{block})?$

$\text{while_stmt} ::= \text{"asLongAs"} \text{expr} \text{block}$

$\text{for_stmt} ::= \text{"for"}$

$\quad \text{"(" for_init ", " expr ", " for_update ")"} \text{"}$

$\quad \text{block}$

$\text{for_init} ::= \text{let_stmt_simple}$

$\text{for_update} ::= \text{set_stmt_simple}$

$\quad | \text{expr}$

$\text{deliver_stmt} ::= \text{"deliver"} \text{expr}?$

$\text{show_stmt} ::= \text{"show"} \text{expr}$

$\text{oops_stmt} ::= \text{"oops"} \text{expr}$

$\text{expr_stmt} ::= \text{expr}$

$\text{block} ::= \{ \text{statement}^* \}$

$\text{expr} ::= \text{expr} ("+" | "-" | "*" | "/" | "%") \text{expr}$

$| \text{expr} ("=" | "!=" | "<" | ">" | "<=" | ">=") \text{expr}$

$| \text{expr} (\text{"and"} | \text{"or"} | \text{"xor"}) \text{expr}$

$| \text{"not"} \text{expr}$

$| "-" \text{expr}$

$| \text{expr} [\text{"expr"}]$

$| \text{expr} "." \text{ID} (\text{"arg_list?"})$

$| \text{expr} "." \text{ID}$

$| \text{ID} (\text{"arg_list?"})$

$| \text{"summon"} \text{ID} (\text{"arg_list?"})$

$| \text{"ohmy"}$

$| \text{"ohmy"} "." \text{ID}$

$| (\text{"expr"})$

$| \text{ID}$

$| \text{INT_LIT}$

$| \text{FLOAT_LIT}$

$| \text{STRING}$

$| \text{"indeed"}$

$| \text{"nope"}$

| "nothing"

$\text{arg_list} ::= \text{expr} ("," \text{expr})^*$

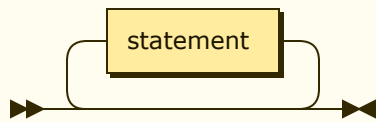
$\text{ID} ::= [\text{a-zA-Z_}][\text{a-zA-Z0-9_}]^*$

$\text{INT_LIT} ::= [0-9]^+$

$\text{FLOAT_LIT} ::= [0-9]^+ "." [0-9]^+$

$\text{STRING} ::= "" [\wedge]^* ""$

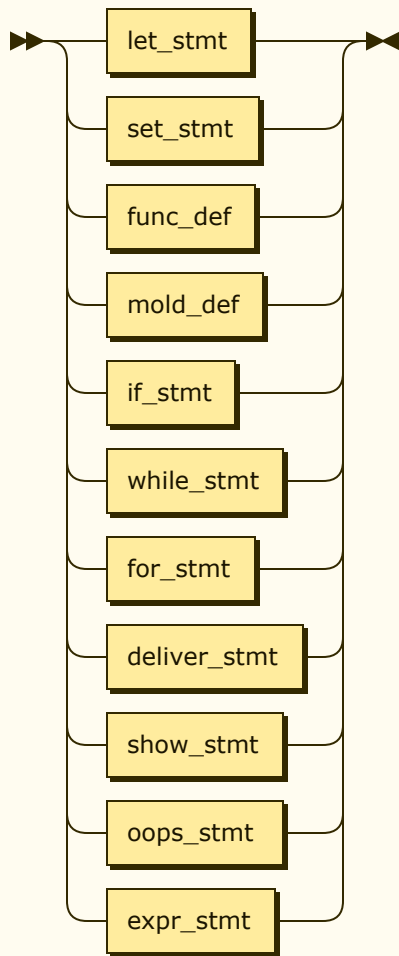
program:



program ::= statement*

no references

statement:

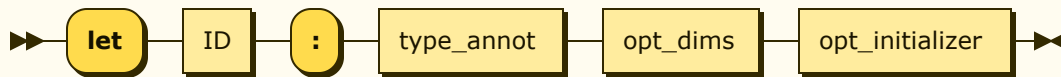


```
statement
  ::= let_stmt
  | set_stmt
  | func_def
  | mold_def
  | if_stmt
  | while_stmt
  | for_stmt
  | deliver_stmt
  | show_stmt
  | oops_stmt
  | expr_stmt
```

referenced by:

- block
- program

let_stmt:

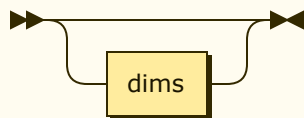


```
let_stmt ::= 'let' ID ':' type_annot opt_dims opt_initializer
```

referenced by:

- mold_member
- statement

opt_dims:

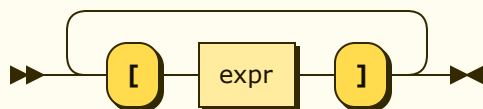


```
opt_dims ::= dims?
```

referenced by:

- let_stmt
- param

dims:

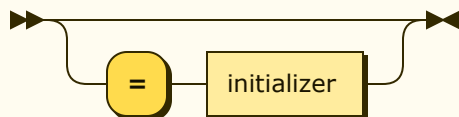


```
dims      ::= ( '[' expr ']' )+
```

referenced by:

- opt_dims

opt_initializer:

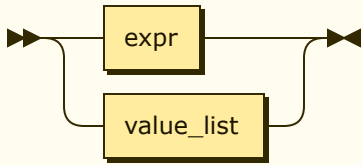


```
opt_initializer
    ::= ( '=' initializer )?
```

referenced by:

- let_stmt

initializer:

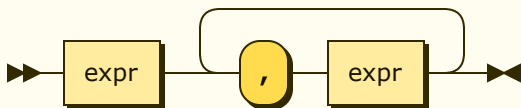


```
initializer  
  ::= expr  
  | value_list
```

referenced by:

- opt_initializer

value_list:

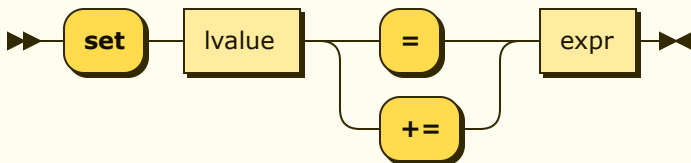


```
value_list  
  ::= expr ( ',' expr )+
```

referenced by:

- initializer

set_stmt:

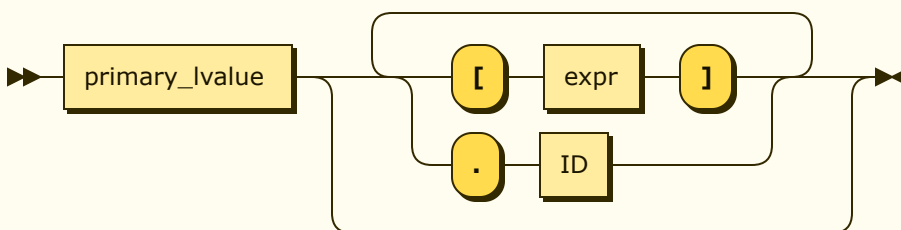


```
set_stmt ::= 'set' lvalue ( '=' | '+=' ) expr
```

referenced by:

- statement

lvalue:

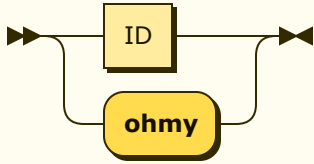


```
lvalue ::= primary_lvalue ( '[' expr ']' | '.' ID )*
```

referenced by:

- set_stmt

primary_lvalue:

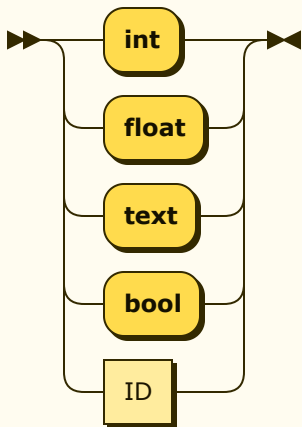


```
primary_lvalue  
  ::= ID  
  | 'ohmy'
```

referenced by:

- lvalue

type_annot:

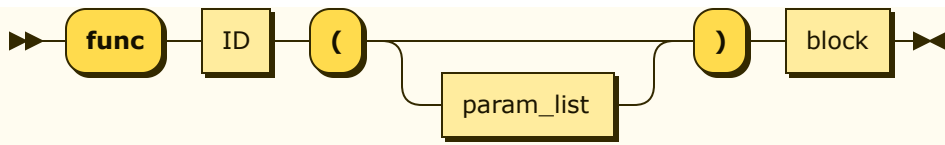


```
type_annot  
  ::= 'int'  
  | 'float'  
  | 'text'  
  | 'bool'  
  | ID
```

referenced by:

- let_stmt
- param

func_def:

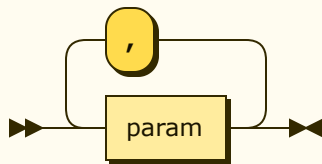


`func_def ::= 'func' ID '(' param_list? ')' block`

referenced by:

- [mold_member](#)
- [statement](#)

param_list:

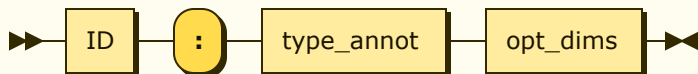


`param_list
::= param (',' param)*`

referenced by:

- [func_def](#)

param:

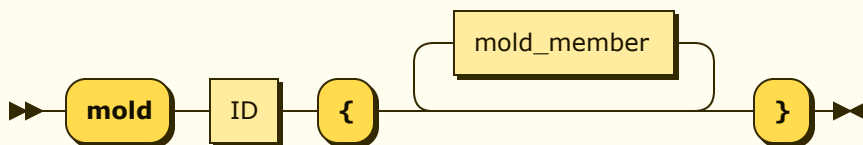


`param ::= ID ':' type_annot opt_dims`

referenced by:

- [param_list](#)

mold_def:

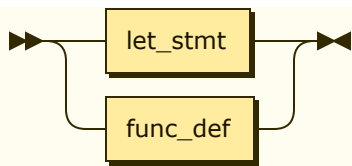


`mold_def ::= 'mold' ID '{' mold_member* '}'`

referenced by:

- [statement](#)

mold_member:



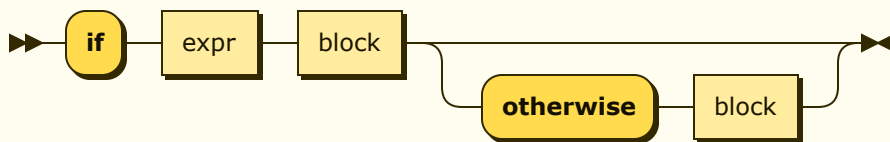
```

mold_member
  ::= let_stmt
    | func_def
  
```

referenced by:

- mold_def

if_stmt:



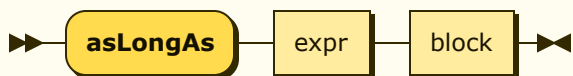
```

if_stmt ::= 'if' expr block ( 'otherwise' block )?
  
```

referenced by:

- statement

while_stmt:



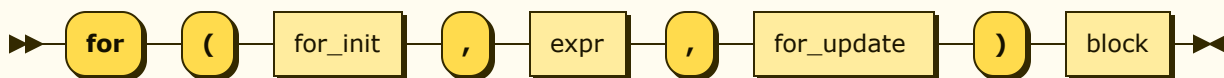
```

while_stmt
  ::= 'asLongAs' expr block
  
```

referenced by:

- statement

for_stmt:



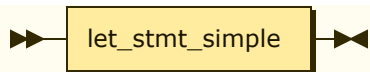
```

for_stmt ::= 'for' '(' for_init ',' expr ',' for_update ')' block
  
```

referenced by:

- statement

for_init:

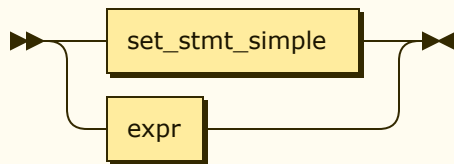


```
for_init ::= let_stmt_simple
```

referenced by:

- for_stmt

for_update:

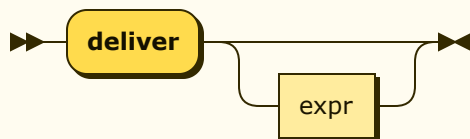


```
for_update  
  ::= set_stmt_simple  
     | expr
```

referenced by:

- for_stmt

deliver_stmt:

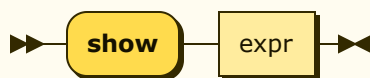


```
deliver_stmt  
  ::= 'deliver' expr?
```

referenced by:

- statement

show_stmt:



```
show_stmt  
  ::= 'show' expr
```

referenced by:

- statement

oops_stmt:



```
oops_stmt  
  ::= 'oops' expr
```

referenced by:

- [statement](#)

expr_stmt:

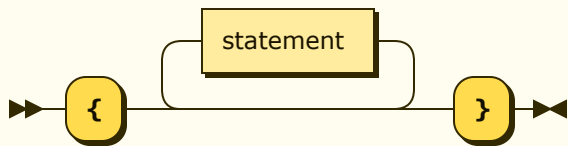


```
expr_stmt  
  ::= expr
```

referenced by:

- [statement](#)

block:

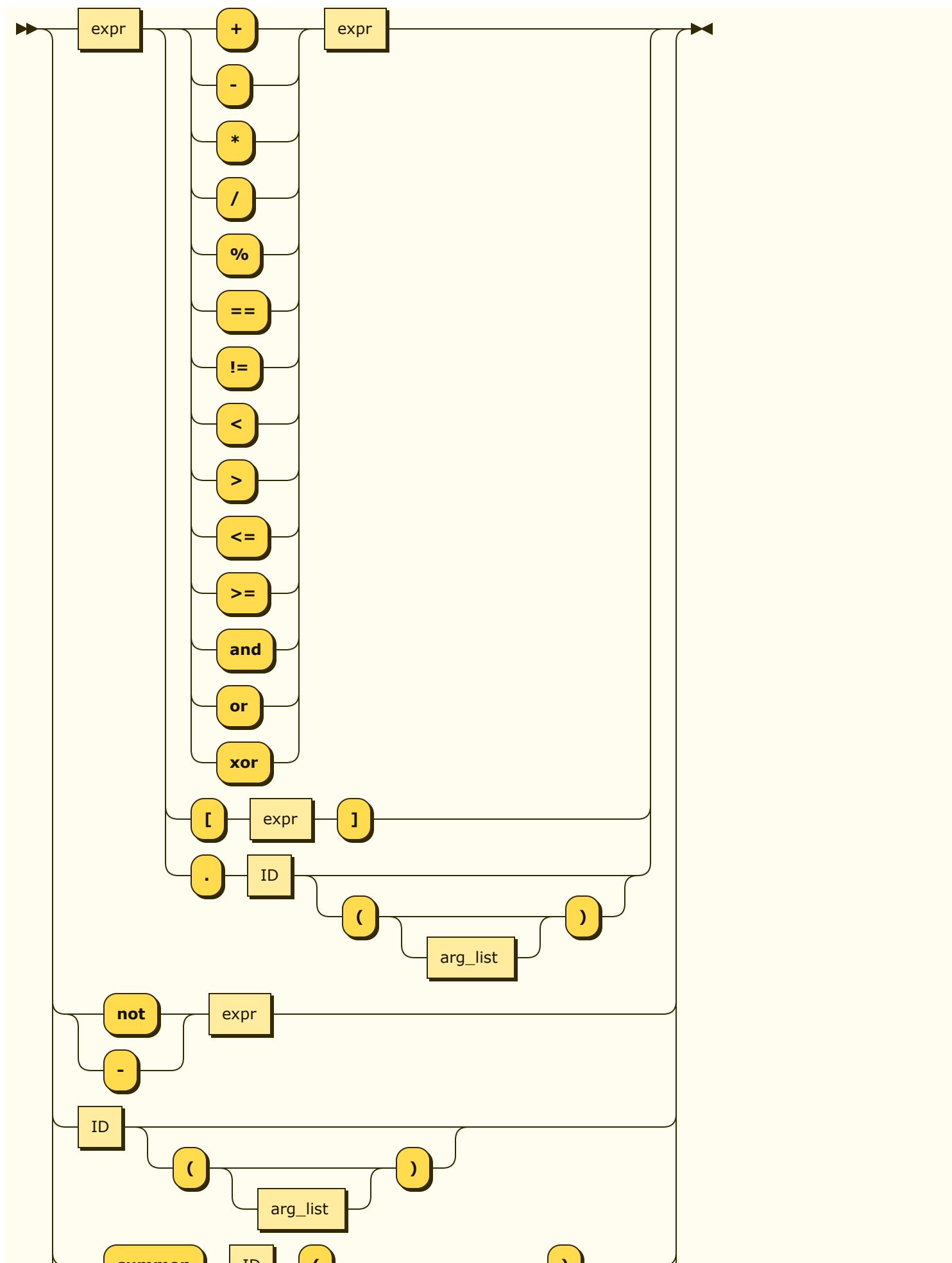


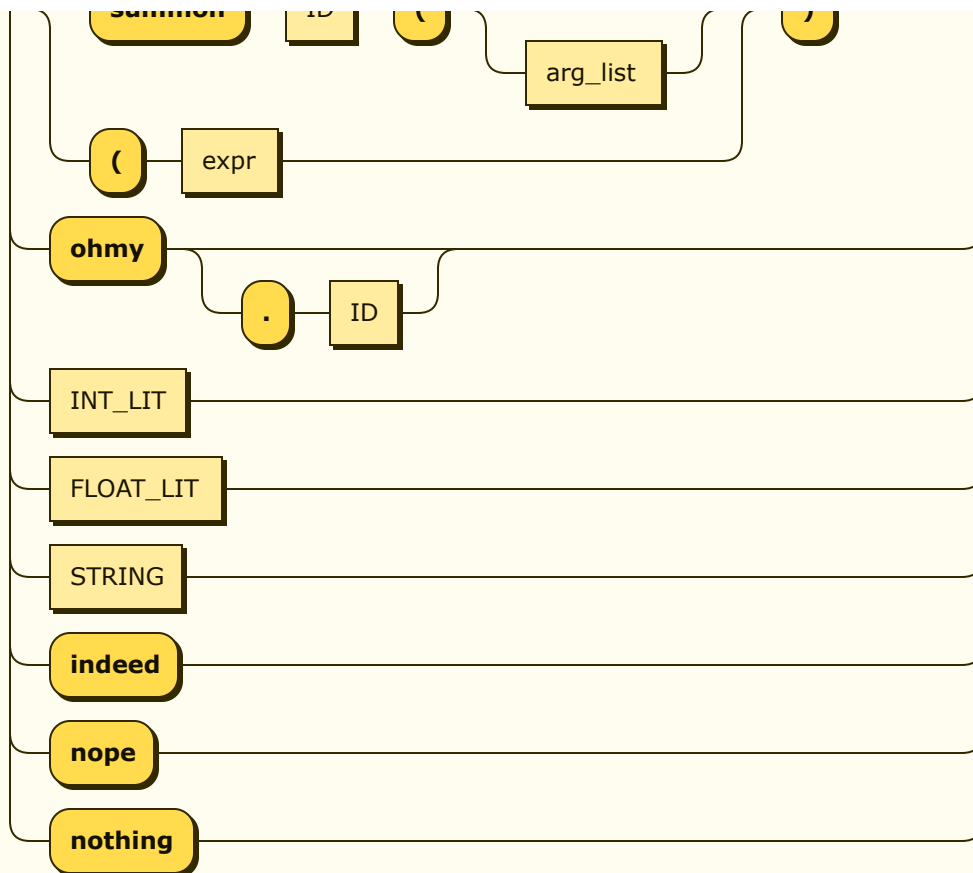
```
block    ::= '{' statement* '}'
```

referenced by:

- [for_stmt](#)
- [func_def](#)
- [if_stmt](#)
- [while_stmt](#)

expr:





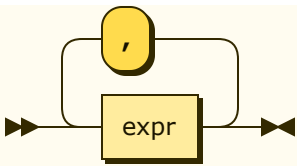
```

expr      ::= expr ( ( '+' | '-' | '*' | '/' | '%' | '==' | '!=' | '<' | '>' | '<=' | '>=' | 'and' | 'or' | 'xor'
                    | arg_list? ')' )? )
            | ( 'not' | '-' ) expr
            | ID ( '(' arg_list? ')' )?
            | ( 'summon' ID '(' arg_list? | '(' expr ) ')'
            | 'ohmy' ( '.' ID )?
            | INT_LIT
            | FLOAT_LIT
            | STRING
            | 'indeed'
            | 'nope'
            | 'nothing'
  
```

referenced by:

- [arg_list](#)
- [deliver stmt](#)
- [dims](#)
- [expr](#)
- [expr stmt](#)
- [for stmt](#)
- [for update](#)
- [if stmt](#)
- [initializer](#)
- [lvalue](#)
- [oops stmt](#)
- [set stmt](#)
- [show stmt](#)
- [value list](#)
- [while stmt](#)

arg_list:

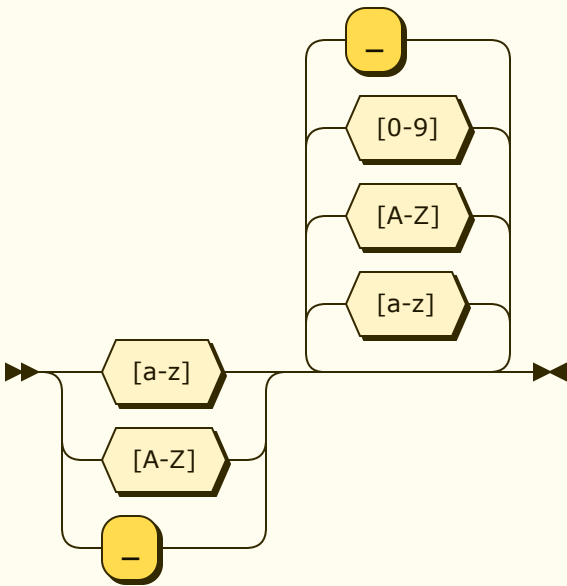


`arg_list ::= expr ( ',' expr )*`

referenced by:

- expr

ID:

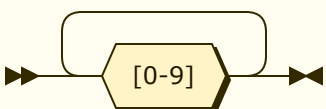


`ID ::= [a-zA-Z_] [a-zA-Z0-9_]*`

referenced by:

- expr
- func_def
- let_stmt
- lvalue
- mold_def
- param
- primary_lvalue
- type_annot

INT_LIT:

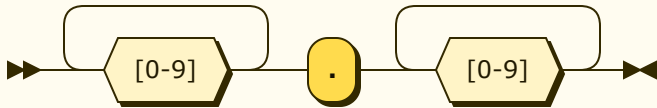


`INT_LIT ::= [0-9]+`

referenced by:

- [expr](#)

FLOAT_LIT:

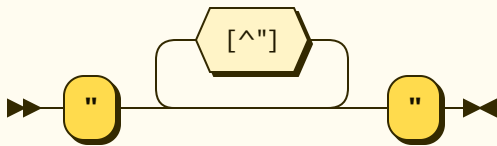


```
FloatLit
FLOAT_LIT
  ::= [0-9]+ '.' [0-9]+
```

referenced by:

- [expr](#)

STRING:



```
String
STRING
  ::= '"' [^"]* '"'
```

referenced by:

- [expr](#)